

A AURORA ESTABELECE OS
PRIMEIROS SISTEMAS DA COLÔNIA

A ESTRUTURA DE
ARMAZENAMENTO
ORGANIZANDO OS
DADOS QUE
SUSTENTAM A
COLÔNIA

2

LISTA DE FIGURAS

Figura 1 – Expressão booleana $X=A+B$ e tabela-verdade com sua respectiva porta lógica.....	12
Figura 2 – Expressão booleana $X=(A+B).C$ e circuito lógico.....	14
Figura 3 – Expressão booleana $X=A+(A+B).C$ e circuito lógico	16
Figura 4 – Circuito lógico da expressão booleana $X=A+(A+B).C$	17
Figura 5 – Construção de lógicas a partir desta tabela-verdade	22
Figura 6 – Expressão lógica da tabela-verdade a ser extraída	23
Figura 7 – Comparação dos termos extraídos pelos mintermos e maxtermos	26
Figura 8 – Identificando os mintermos desta tabela-verdade	27
Figura 9 – Identificando os mintermos desta tabela-verdade e negando os zeros ...	28
Figura 10 – Extração da expressão pelos mintermos, soma de produtos.....	29
Figura 11 – Forma canônica “Soma de Produtos”	30
Figura 12 – Extração da expressão booleana por maxtermos	31
Figura 13 – Extração dos termos por maxtermos, negando os 1s	32
Figura 14 – Extração da expressão pelos maxtermos, produto de somas	33
Figura 15 – Forma canônica “Produto de Somas”	34
Figura 16 – Equivalência lógica entre mintermos e maxtermos	35
Figura 17 – Forma equivalente mais simples	37
Figura 18 – Eliminação de termos repetidos (lei da idempotência)	39
Figura 19 – Aplicação da lei distributiva	40
Figura 20 – Fatoração da variável comum	40
Figura 21 – Simplificação algébrica utilizando as leis da álgebra de Boole	42
Figura 22 – Simplificação algébrica por transformações equivalentes e fatoração sucessiva.....	46
Figura 23 – Extração da expressão booleana por mintermos	50
Figura 24 – Extração da expressão booleana por mintermos, expressão completa .	51
Figura 25 – Simplificação algébrica da expressão $X=(A \cdot B')+(A \cdot B)$	52
Figura 26 – Extração da expressão booleana por maxtermos (1).....	54
Figura 27 – Extração da expressão booleana por maxtermos (2).....	55
Figura 28 – Simplificação algébrica da expressão $X=(A \cdot B) + (A \cdot B')$	56

SUMÁRIO

LÓGICA BOOLEANA E SIMPLIFICAÇÃO ALGÉBRICA.....	5
1 INTRODUÇÃO À LÓGICA BOOLEANA.....	5
1.1 Relação entre expressões booleanas, circuitos lógicos e problemas do mundo real	7
1.2 Importância da simplificação de circuitos lógicos gerados por tabelas-verdade ..	8
1.3 A álgebra de Boole como base para a simplificação de circuitos lógicos.....	9
1.4 Funções booleanas e o conjunto de constantes da lógica booleana	10
2 EXPRESSÕES BOOLEANAS E REPRESENTAÇÃO POR TABELAS-VERDADE	12
2.1 Análise da expressão booleana, variáveis de entrada e saída.....	12
2.2 Análise da expressão booleana composta e sua tabela-verdade	14
2.3 Precedência de operadores em expressões booleanas e análise da tabela-verdade	16
2.4 Propriedades fundamentais da álgebra de Boole e o comportamento dos termos booleanos.....	18
2.5 Construção de lógicas a partir de tabelas-verdade	21
2.6 Extração de expressões booleanas a partir da tabela-verdade.....	22
2.7 Mintermos e maxtermos, extração de expressões booleanas a partir da tabela-verdade	24
2.7.1 Mintermos.....	24
2.7.2 Maxtermos.....	25
2.7.3 Relação entre mintermos, maxtermos e a tabela-verdade	25
2.8 Importância acadêmica e prática.....	26
2.9 Extração da expressão booleana por mintermos (forma soma de produtos)	27
2.9.1 Identificação dos mintermos a partir da saída igual a 1	27
2.9.2 Por que negar variáveis quando a entrada é 0.....	28
2.9.3 Uso do operador AND na construção de cada mintermo	29
2.9.4 Formação da expressão final com o operador OR	29
2.9.5 Significado da forma “soma de produtos”	30
2.10 Extração da expressão booleana por meio de maxtermos (forma produto de somas)	31
2.10.1 Identificação dos maxtermos a partir da saída igual a 0.....	31
2.10.2 Por que as variáveis não são negadas quando a entrada é 0?.....	32
2.10.3 Uso do operador OR na construção de cada maxtermo	32
2.10.4 Formação da expressão final com o operador AND.....	33
2.10.5 Significado da forma “produto de somas”	34
2.11 Equivalência lógica entre mintermos e maxtermos	34
2.11.1 Verificação por meio da tabela-verdade	35
2.11.2 Interpretação conceitual	36
3 SIMPLIFICAÇÃO ALGÉBRICA	36
3.1 Simplificação algébrica com eliminação de redundâncias e reorganização de termos	38
3.1.1 Reorganização dos termos (lei comutativa)	39
3.1.2 Eliminação de termos repetidos (lei da idempotência)	39
3.1.3 Aplicação da lei distributiva	39
3.1.4 Nova aplicação da lei da idempotência	40

3.1.5 Fatoração da variável comum	40
3.1.6 Aplicação da lei da dominação	41
3.1.7 Aplicação da lei da identidade	41
3.1.8 Interpretação conceitual da simplificação	41
3.2 Exemplo de simplificação algébrica utilizando as leis da álgebra de Boole	42
3.2.1 Aplicação da lei distributiva	43
3.2.2 Aplicação da lei distributiva	43
3.2.3 Eliminação de redundância (lei da idempotência no OR)	43
3.2.4 Fatoração da variável comum	44
3.2.5 Justificativa de que $(1 + B = 1)$	44
3.2.6 Justificativa de que $(A \cdot 1 = A)$	44
3.2.7 Interpretação conceitual da simplificação	45
3.3 Simplificação algébrica por transformações equivalentes e fatoração sucessiva	45
3.3.1 Eliminação de termos repetidos (lei da idempotência)	46
3.3.2 Transformação algébrica equivalente	46
3.3.3 Aplicação da lei distributiva	47
3.3.4 Fatoração sucessiva de termos comuns	47
3.3.5 Fatoração final e aplicação das leis da álgebra de Boole	48
3.3.6 Resultado final da simplificação	48
3.3.7 Interpretação conceitual do exemplo	48
3.4 Extração da expressão por mintermos e simplificação algébrica	49
3.4.1 Leitura da tabela-verdade e identificação das linhas relevantes	49
3.4.2 Construção dos mintermos (por que negar quando a entrada é 0?)	50
3.4.3 Por que o operador AND é usado dentro do mintermo	51
3.4.4 Formação da expressão final (soma de produtos)	51
3.4.5 Simplificação algébrica da expressão obtida	52
3.4.6 Interpretação do resultado	53
3.5 Inter extração da expressão por maxtermos e simplificação algébrica (exemplo completo)	53
3.5.1 Leitura da tabela-verdade e identificação das linhas relevantes	53
3.5.2 Construção dos maxtermos (por que negar quando a entrada é 1?)	54
3.5.3 Por que o operador OR é usado dentro do maxtermo	55
3.5.4 Formação da expressão final (produto de somas)	55
3.5.5 Simplificação algébrica da expressão obtida	56
3.5.6 Comparação final: igualdade entre as simplificações por mintermos e por maxtermos	58
CONCLUSÃO	59
REFERÊNCIAS	60
GLOSSÁRIO	62

LÓGICA BOOLEANA E SIMPLIFICAÇÃO ALGÉBRICA

1 INTRODUÇÃO À LÓGICA BOOLEANA

A lógica booleana constitui um dos **pilares fundamentais da ciência da computação** e da **engenharia de sistemas digitais**, servindo como base teórica para a representação formal de decisões, processos lógicos e mecanismos de controle presentes em praticamente todos os sistemas computacionais modernos. Seu estudo é essencial para a compreensão do funcionamento interno dos computadores, dos circuitos eletrônicos digitais, dos sistemas embarcados e, de forma mais abstrata, dos processos de tomada de decisão fundamentados em lógica formal e raciocínio matemático.

Formalizada no século XIX por **George Boole**, a lógica booleana surgiu inicialmente como um **sistema matemático abstrato** destinado a modelar o raciocínio humano por meio de símbolos, variáveis e operações algébricas. A principal inovação proposta por Boole foi a substituição de proposições verbais por **expressões matemáticas**, permitindo que afirmações lógicas fossem manipuladas de maneira sistemática, rigorosa e independente da linguagem natural. Essa abordagem inaugurou um novo paradigma na lógica matemática, no qual o pensamento lógico passou a ser tratado como um sistema formal passível de cálculo.

Diferentemente da álgebra tradicional, que opera sobre um conjunto infinito de valores numéricos e tem como foco grandezas contínuas, a álgebra booleana trabalha com um **conjunto discreto e finito**, composto exclusivamente por dois valores possíveis. Esses valores são comumente representados como **0 e 1**, **falso e verdadeiro**, ou ainda **desligado e ligado**, dependendo do contexto de aplicação. Essa característica binária torna a lógica booleana particularmente adequada à modelagem de sistemas digitais, nos quais os estados físicos dos componentes eletrônicos assumem naturalmente apenas dois níveis bem definidos.

Com o advento da computação eletrônica no século XX, especialmente após o desenvolvimento dos primeiros computadores digitais, a lógica booleana passou a desempenhar um papel central na construção e no funcionamento das máquinas computacionais. Seus princípios mostraram-se perfeitamente alinhados ao

comportamento físico dos dispositivos eletrônicos digitais, como **válvulas, transistores e circuitos integrados**, que operam por meio de sinais elétricos binários. Cada operação lógica definida pela álgebra booleana pode ser implementada fisicamente por meio de **portas lógicas**, tornando essa teoria a **ponte conceitual entre o pensamento abstrato e a execução física da computação**.

No âmbito da ciência da computação, a lógica booleana ultrapassa o domínio do hardware e se manifesta de forma direta em diversas camadas do software. Estruturas condicionais em linguagens de programação, expressões lógicas em consultas a bancos de dados, mecanismos de decisão em algoritmos, regras de inferência em sistemas especialistas e modelos de raciocínio em inteligência artificial são todos fundamentados nos princípios da lógica booleana. Dessa forma, compreender essa lógica não significa apenas entender circuitos digitais, mas também adquirir a capacidade de **modelar problemas computacionais de forma precisa e formal**.

A lógica booleana é amplamente aplicada em diversas áreas da computação, destacando-se:

- Projeto, análise e otimização de circuitos digitais;
- Arquitetura e organização de computadores;
- Linguagens de programação e controle de fluxo por estruturas condicionais;
- Banco de dados, especialmente na formulação de consultas lógicas;
- Inteligência artificial, sistemas especialistas e motores de inferência;
- Otimização de algoritmos e simplificação de expressões lógicas e computacionais.

Além de sua relevância prática, a lógica booleana possui grande importância do ponto de vista teórico, pois fornece ao estudante uma base sólida para o desenvolvimento do **pensamento lógico-formal**, competência essencial para a resolução de problemas complexos em ciência da computação. A capacidade de abstrair situações reais em termos de variáveis lógicas, relações e operações formais é uma habilidade central na formação do profissional da área.

Dessa forma, este capítulo tem como objetivo apresentar de maneira sistemática os **fundamentos da lógica booleana**, suas operações básicas,

propriedades algébricas e principais técnicas de simplificação. Ao final do estudo, espera-se que o estudante seja capaz de **modelar, analisar e otimizar expressões lógicas**, compreendendo tanto seu significado matemático quanto suas implicações práticas na construção de sistemas computacionais eficientes, corretos e confiáveis.

1.1 Relação entre expressões booleanas, circuitos lógicos e problemas do mundo real

Os circuitos lógicos podem ser compreendidos como a **materialização física das expressões booleanas**. Cada circuito digital é projetado para executar, de forma automática e determinística, uma ou mais expressões da álgebra booleana, traduzindo operações lógicas abstratas em comportamentos físicos observáveis. Dessa forma, pode-se afirmar que os circuitos lógicos **executam expressões booleanas**, assim como um programa executa um algoritmo.

As expressões booleanas, por sua vez, são construções matemáticas que representam relações lógicas entre variáveis binárias. Essas variáveis normalmente correspondem a **condições, estados ou decisões** presentes em problemas do mundo real. Exemplos incluem a verificação de sensores em um sistema embarcado, a validação de permissões em um sistema computacional ou a avaliação de múltiplos critérios em um processo decisório automatizado.

Ao projetar um circuito lógico, o engenheiro ou cientista da computação parte de um **problema real**, o qual é inicialmente modelado de forma abstrata por meio de variáveis booleanas e operadores lógicos. Em seguida, essa modelagem é expressa matematicamente por uma **expressão booleana**, que descreve formalmente o comportamento desejado do sistema. Finalmente, essa expressão é implementada fisicamente por meio da interconexão de portas lógicas, resultando em um circuito capaz de responder corretamente às diferentes combinações de entradas possíveis.

Essa correspondência direta entre **problema real** → **expressão booleana** → **circuito lógico** evidencia o papel central da lógica booleana como elo entre a abstração matemática e a implementação física. Qualquer sistema digital, independentemente de sua complexidade, pode ser decomposto em um conjunto de expressões booleanas interligadas, que operam de forma conjunta para produzir o comportamento global do sistema.

Portanto, compreender como circuitos lógicos executam expressões booleanas não é apenas um exercício teórico, mas uma competência fundamental para o desenvolvimento de sistemas computacionais confiáveis, eficientes e corretos. Essa compreensão permite ao estudante analisar, projetar e otimizar circuitos e sistemas digitais, assegurando que a solução implementada corresponda fielmente ao problema real que se deseja resolver.

1.2 Importância da simplificação de circuitos lógicos gerados por tabelas-verdade

Circuitos lógicos podem ser obtidos diretamente a partir de **tabelas-verdade**, uma vez que estas descrevem de forma completa o comportamento lógico desejado para todas as combinações possíveis das variáveis de entrada. Embora esse método seja conceitualmente simples e sistemático, os circuitos resultantes tendem a apresentar uma **estrutura excessivamente complexa**, caracterizada por um número elevado de portas lógicas e interconexões.

Na prática, observa-se que muitos desses circuitos admitem **simplificações significativas**, pois diferentes combinações de entradas podem levar ao mesmo resultado lógico, permitindo a eliminação de termos redundantes nas expressões booleanas. A aplicação de técnicas de simplificação algébrica possibilita a redução do número de operações lógicas necessárias para representar uma determinada função, sem alterar o comportamento funcional do sistema.

A redução do número de portas lógicas impacta diretamente diversos aspectos do sistema digital. Do ponto de vista físico, circuitos mais simples demandam **menor quantidade de componentes**, menos conexões e menor área de implementação em circuitos integrados. Isso resulta em **menor custo de produção, menor consumo de energia e maior confiabilidade**, uma vez que sistemas menos complexos estão menos sujeitos a falhas.

Além disso, a simplificação dos circuitos reduz o **grau de dificuldade na montagem e manutenção**, facilitando o processo de projeto, teste e depuração. Em sistemas digitais de maior porte, essa redução de complexidade é essencial para garantir escalabilidade, desempenho adequado e viabilidade econômica.

Sob a perspectiva da ciência da computação, a simplificação de expressões booleanas também possui um valor conceitual relevante, pois estimula o desenvolvimento do **pensamento algorítmico e da abstração lógica**. Ao identificar padrões, redundâncias e equivalências lógicas, o estudante aprende a transformar soluções corretas em soluções **mais eficientes**, habilidade fundamental tanto no projeto de hardware quanto no desenvolvimento de software.

Dessa forma, a simplificação algébrica de circuitos lógicos não deve ser vista apenas como uma etapa opcional do projeto, mas como um **princípio fundamental da otimização de sistemas digitais**, contribuindo para a construção de soluções mais eficientes, econômicas e tecnicamente robustas.

1.3 A álgebra de Boole como base para a simplificação de circuitos lógicos

O estudo da simplificação de circuitos lógicos está intrinsecamente relacionado ao conhecimento da **álgebra de Boole**, uma vez que toda técnica de redução e otimização de circuitos digitais fundamenta-se em suas definições, propriedades e leis formais. Sem o domínio dessa álgebra, torna-se inviável compreender de forma sistemática como expressões lógicas podem ser transformadas e simplificadas sem alteração do comportamento funcional do sistema.

A **lógica booleana** pode ser definida como um sistema lógico-matemático que opera sobre valores binários e estabelece regras formais para a combinação e manipulação desses valores. Ela fornece os princípios conceituais que permitem representar proposições lógicas, condições e decisões por meio de variáveis e operadores bem definidos. A álgebra de Boole, por sua vez, constitui a formalização matemática dessa lógica, oferecendo um conjunto de operações, axiomas e propriedades que possibilitam o tratamento algébrico das expressões booleanas.

Diferentemente da álgebra numérica tradicional, a álgebra de Boole não está voltada ao cálculo de grandezas, mas à **análise de relações lógicas**. Suas variáveis assumem apenas dois estados possíveis, geralmente associados aos valores verdadeiro e falso, ou 1 e 0. A partir desses estados, são definidas operações fundamentais, como conjunção, disjunção e negação, que permitem construir expressões capazes de descrever o comportamento lógico de sistemas digitais.

No contexto da simplificação de circuitos lógicos, a álgebra de Boole fornece as ferramentas necessárias para identificar **equivalências lógicas**, **termos redundantes** e **estruturas passíveis de redução**. Por meio da aplicação de leis como identidade, anulação, idempotência, complementaridade e distributividade, é possível transformar expressões complexas em formas mais simples e eficientes, preservando integralmente sua funcionalidade.

Assim, o domínio da álgebra de Boole viabiliza a simplificação de circuitos lógicos e promove uma compreensão mais profunda da relação entre a **representação matemática** de um sistema e sua **implementação física**. Essa base teórica é indispensável para que o estudante de ciência da computação desenvolva a capacidade de projetar, analisar e otimizar sistemas digitais de maneira rigorosa, eficiente e confiável.

1.4 Funções booleanas e o conjunto de constantes da lógica booleana

As **funções booleanas** constituem o elemento central da lógica booleana e da álgebra de Boole, pois são elas que descrevem formalmente o comportamento lógico de sistemas digitais. Uma função booleana pode ser definida como uma relação matemática que associa um conjunto de variáveis booleanas de entrada a um único valor booleano de saída, de acordo com regras lógicas previamente estabelecidas.

Formalmente, uma função booleana recebe variáveis que assumem valores pertencentes ao conjunto $\{0, 1\}$ e produz como resultado um valor que também pertence a esse mesmo conjunto. Dessa forma, uma função booleana pode ser representada por meio de **expressões algébricas**, **tabelas-verdade** ou **circuitos lógicos**, sendo essas representações equivalentes entre si. Cada combinação possível dos valores das variáveis de entrada corresponde a um resultado lógico específico, o que torna as funções booleanas particularmente adequadas para modelar decisões, condições e comportamentos discretos.

No contexto dos sistemas digitais, as funções booleanas descrevem como sinais de entrada são processados para gerar sinais de saída. Sensores, controles, comparações e mecanismos de decisão presentes em sistemas computacionais podem ser abstraídos como funções booleanas, nas quais cada variável representa uma condição lógica relevante do sistema.

Operando exclusivamente com **duas constantes booleanas**, tradicionalmente representadas por:

- 0 (zero);
- 1 (um).

A existência de apenas duas constantes não é uma limitação arbitrária, mas sim uma consequência direta da natureza da lógica booleana e dos sistemas que ela modela. Essas constantes representam os dois estados lógicos fundamentais possíveis em um sistema binário: falso e verdadeiro, ausência e presença de sinal, desligado e ligado. Essa dualidade reflete tanto uma escolha matemática quanto uma adequação prática à realidade física dos sistemas digitais.

Do ponto de vista matemático, a lógica booleana é uma lógica **bivalente**, ou seja, toda proposição lógica deve assumir exatamente um de dois valores possíveis. Essa característica garante simplicidade formal, consistência lógica e facilidade de manipulação algébrica. A introdução de mais valores comprometeria a estrutura da álgebra booleana clássica e exigiria sistemas lógicos mais complexos, como as lógicas multivaloradas, que fogem ao escopo dos sistemas digitais tradicionais.

Sob a perspectiva física, a utilização de apenas duas constantes está diretamente relacionada à implementação eletrônica dos circuitos digitais. Componentes como transistores operam de forma confiável em dois estados bem definidos, correspondentes a níveis distintos de tensão elétrica. Essa característica torna os sistemas binários mais robustos contra ruídos, variações ambientais e imprecisões físicas, justificando a adoção universal do modelo binário na computação.

Assim, todas as funções booleanas são construídas exclusivamente a partir dessas duas constantes fundamentais e das operações lógicas definidas pela álgebra de Boole. A combinação dessas constantes com variáveis e operadores permite representar qualquer comportamento lógico necessário para a implementação de sistemas computacionais, desde circuitos simples até arquiteturas digitais altamente complexas.

A compreensão das funções booleanas e do papel das constantes 0 e 1 é, portanto, essencial para o estudo da lógica booleana, pois estabelece a base conceitual sobre a qual se apoiam as expressões lógicas, os circuitos digitais e as técnicas de simplificação algébrica abordadas ao longo deste capítulo.

2 EXPRESSÕES BOOLEANAS E REPRESENTAÇÃO POR TABELAS-VERDADE

As **expressões booleanas** descrevem relações lógicas entre variáveis que assumem apenas dois valores: **0** e **1**. Elas são formadas com operadores como **OR(+)**, **AND(·)** e **NOT ($\bar{A}$)**.

A **tabela-verdade** é utilizada para representar o comportamento de uma expressão, mostrando todas as combinações possíveis das entradas e o valor da saída correspondente. Para n variáveis, a tabela possui 2^n linhas.

Essa representação permite analisar, validar e comparar expressões lógicas, sendo fundamental no estudo e no projeto de sistemas digitais.

2.1 Análise da expressão booleana, variáveis de entrada e saída

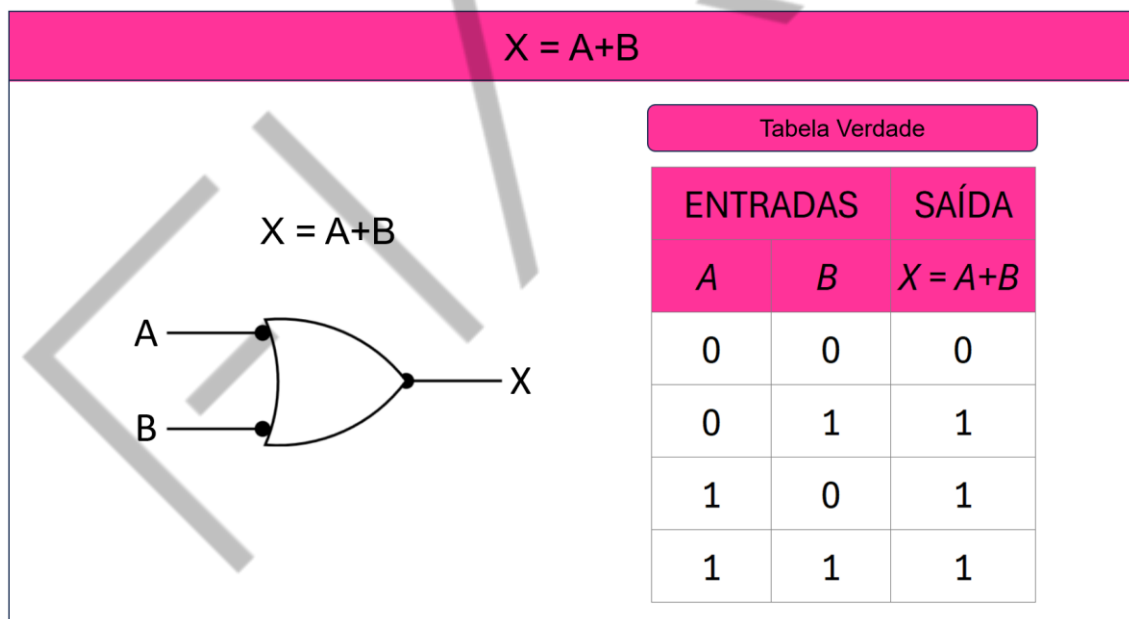


Figura 1 – Expressão booleana $X=A+B$ e tabela-verdade com sua respectiva porta lógica
Fonte: Elaborada pelo autor (2026)

A expressão booleana acima apresentada pela porta lógica OR é dada por:

$$X = A + B$$

Nessa expressão, **A** e **B** representam **variáveis booleanas de entrada**, enquanto **X** representa a **variável booleana de saída**. Cada variável booleana pode

assumir exclusivamente um dos dois valores possíveis do conjunto $\{0, 1\}$, os quais correspondem, respectivamente, aos estados lógicos **falso** e **verdadeiro**.

O operador “+” utilizado na expressão não corresponde à soma aritmética convencional, mas sim à operação lógica **OU (OR)** da álgebra de Boole. Essa operação estabelece que a saída da função será igual a **1** sempre que **ao menos uma** das variáveis de entrada for igual a 1. O resultado será igual a **0** apenas quando **todas** as entradas assumirem o valor 0.

A tabela-verdade associada à expressão descreve de forma completa o comportamento lógico da função, apresentando todas as combinações possíveis dos valores das variáveis **A** e **B**, bem como o valor correspondente da saída **X**. Observe-se que:

- Quando **A = 0** e **B = 0**, a saída **X = 0**, pois nenhuma condição verdadeira está presente;
- Quando **A = 0** e **B = 1**, a saída **X = 1**, pois ao menos uma entrada é verdadeira;
- Quando **A = 1** e **B = 0**, a saída **X = 1**, pelo mesmo princípio;
- Quando **A = 1** e **B = 1**, a saída **X = 1**, uma vez que ambas as entradas são verdadeiras.

Essa representação evidencia a correspondência direta entre a **expressão booleana**, a **tabela-verdade** e o comportamento lógico do sistema. A tabela-verdade constitui, portanto, uma ferramenta fundamental para a análise, validação e implementação de funções booleanas, permitindo verificar de maneira sistemática se a expressão lógica atende aos requisitos do problema que se deseja modelar.

2.2 Análise da expressão booleana composta e sua tabela-verdade

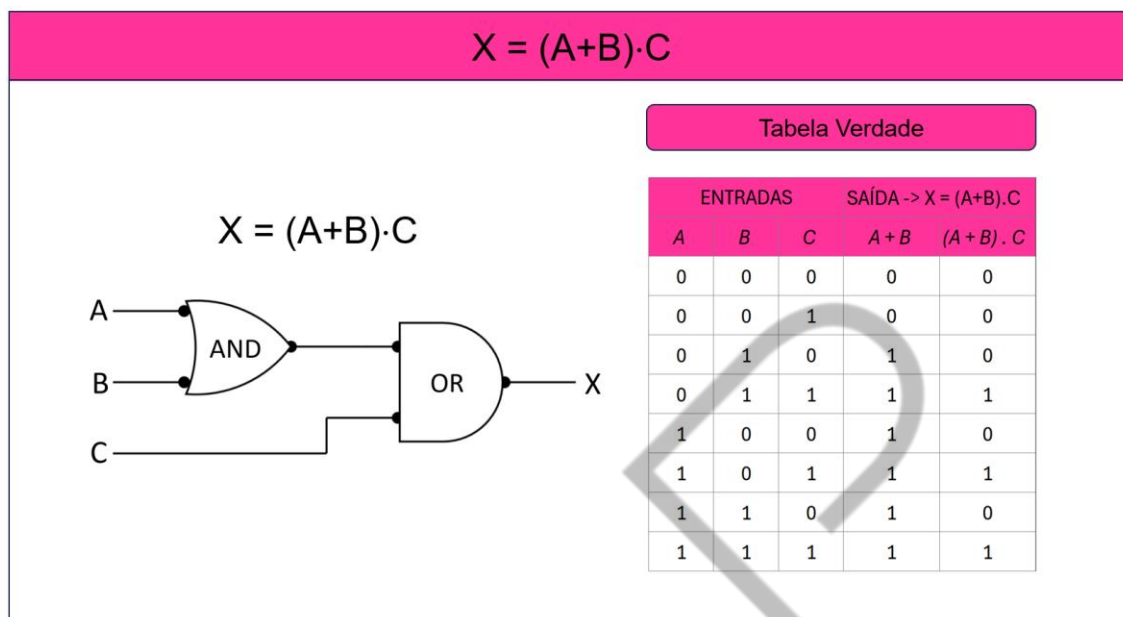


Figura 2 – Expressão booleana $X=(A+B) \cdot C$ e circuito lógico
Fonte: Elaborado pelo autor (2026)

A expressão booleana apresentada acima é:

$$X = (A + B) \cdot C$$

Trata-se de uma **expressão booleana composta**, formada pela combinação de duas operações lógicas fundamentais da álgebra de Boole: a **disjunção lógica (OR)**, representada pelo operador “+”, e a **conjunção lógica (AND)**, representada pelo operador “.”. As variáveis **A**, **B** e **C** são variáveis booleanas de entrada, enquanto **X** representa a variável booleana de saída.

A expressão pode ser interpretada de forma sequencial. Inicialmente, avalia-se a subexpressão **(A + B)**, que corresponde à operação lógica OR entre as variáveis **A** e **B**. O resultado dessa operação será igual a **1** sempre que pelo menos uma dessas variáveis assumir o valor **1**. Em seguida, o resultado obtido é combinado com a variável **C** por meio da operação lógica AND, assim, a saída **X** será igual a 1 apenas quando **C = 1** e, simultaneamente, **A ou B** forem iguais a 1.

A tabela-verdade associada à expressão descreve de forma completa o comportamento lógico da função para todas as combinações possíveis das variáveis de entrada. Observa-se que a tabela apresenta, além da saída final **(A + B) · C**, uma

coluna intermediária correspondente ao valor da subexpressão $A + B$, o que facilita a compreensão do processo lógico envolvido.

A análise da tabela-verdade permite destacar os seguintes comportamentos:

- Quando $C = 0$, a saída X é sempre igual a 0, independentemente dos valores de A e B , pois a operação **AND** exige que todas as entradas sejam verdadeiras para produzir saída 1;
- Quando $C = 1$, a saída X será igual a 1 somente se $A + B = 1$, ou seja, se pelo menos uma das variáveis A ou B for igual a 1;
- A saída X assume o valor 1 exclusivamente nas situações em que a condição lógica composta é plenamente satisfeita.

Essa expressão exemplifica de forma clara como funções booleanas mais complexas podem ser construídas a partir da **combinação de operações lógicas básicas**, permitindo a modelagem precisa de condições e decisões presentes em sistemas computacionais e digitais. A utilização da tabela-verdade, nesse contexto, é fundamental para validar o comportamento da expressão e assegurar que o resultado obtido corresponde corretamente à lógica desejada.

O estudo de expressões compostas como $X = (A + B) \cdot C$ é essencial para a compreensão do funcionamento de circuitos digitais mais elaborados e constitui a base para as técnicas de **simplificação algébrica**, que visam reduzir a complexidade dessas expressões sem alterar sua funcionalidade.

2.3 Precedência de operadores em expressões booleanas e análise da tabela-verdade

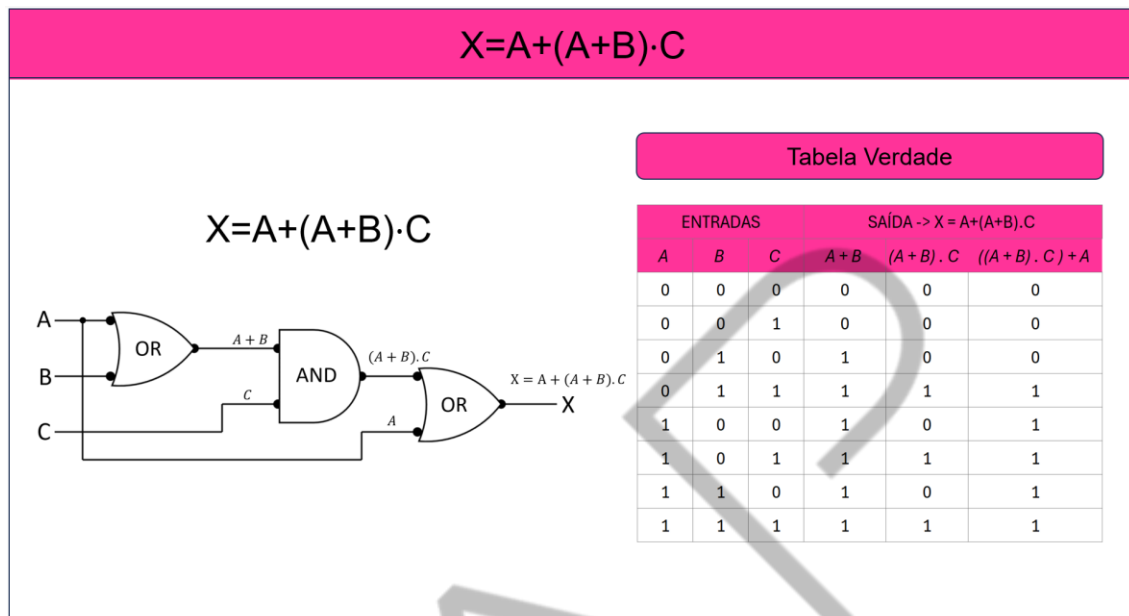


Figura 3 – Expressão booleana $X = A + (A + B) \cdot C$ e circuito lógico
 Fonte: Elaborado pelo autor (2026)

A expressão booleana analisada nesta seção é:

$$X = A + (A + B) \cdot C$$

Essa expressão ilustra de forma clara a importância da **ordem de precedência dos operadores lógicos** na álgebra de Boole. Assim como ocorre na matemática tradicional, as expressões booleanas seguem regras bem definidas que determinam a sequência correta de avaliação das operações, garantindo que o resultado lógico obtido seja único e consistente.

A resolução da expressão segue, obrigatoriamente, a seguinte ordem:

1. Resolução das expressões entre parênteses;
2. Aplicação da operação lógica AND ($\cdot$);
3. Aplicação da operação lógica OR ($+$).

Inicialmente, avalia-se a subexpressão **(A + B)**, que corresponde à operação lógica OR entre as variáveis **A** e **B**. Esse resultado intermediário é apresentado

explicitamente na tabela-verdade, permitindo observar em quais combinações de entrada a subexpressão assume o valor lógico 1.

Em seguida, o valor obtido em $(A + B)$ é combinado com a variável C por meio da operação lógica **AND**, resultando na expressão intermediária $(A + B) \cdot C$. Essa operação impõe uma condição adicional, mesmo que $A + B = 1$, a saída dessa etapa será igual a 1 apenas se $C = 1$, caso contrário, o resultado será 0.

Por fim, o resultado da operação $(A + B) \cdot C$ é combinado com a variável A por meio da operação lógica **OR**, produzindo o valor final da saída X . Nessa etapa, observa-se que a saída X será igual a 1 sempre que $A = 1$, independentemente dos valores de B e C , ou quando $A = 0$ e $(A + B) \cdot C = 1$.

A tabela-verdade associada à expressão apresenta, de forma detalhada, cada uma dessas etapas de avaliação, incluindo colunas intermediárias que evidenciam o processo lógico. Essa decomposição é fundamental para compreender como expressões booleanas mais complexas são avaliadas e para evitar interpretações incorretas decorrentes da aplicação inadequada da precedência dos operadores.

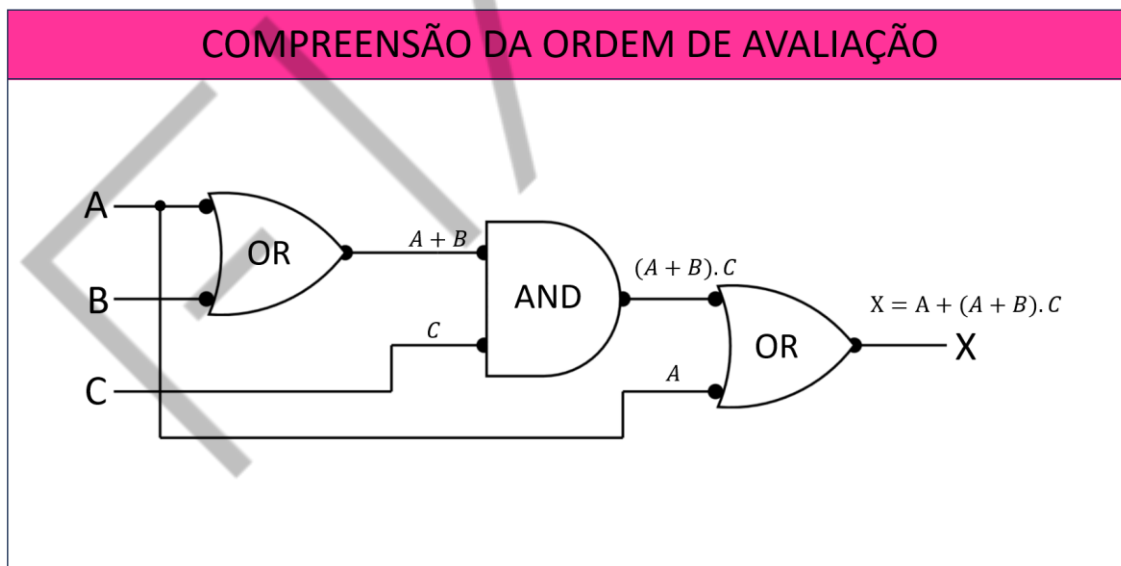


Figura 4 – Circuito lógico da expressão booleana $X = A + (A + B) \cdot C$

Fonte: Elaborado pelo autor (2026)

A correta compreensão da ordem de avaliação é **parênteses, AND e OR**. É essencial tanto para a análise teórica quanto para a implementação prática de circuitos digitais e algoritmos computacionais. Erros na interpretação da precedência podem resultar em comportamentos lógicos inesperados, comprometendo a funcionalidade do sistema. Dessa forma, o domínio dessas regras constitui um

conhecimento fundamental para o projeto e a simplificação de expressões booleanas em ciência da computação.

2.4 Propriedades fundamentais da álgebra de Boole e o comportamento dos termos booleanos

A álgebra de Boole descreve o comportamento de **condições lógicas** e **estados binários**, e **não de quantidades numéricas**. Por essa razão, suas operações apresentam resultados que diferem da álgebra aritmética tradicional. Cada expressão a seguir pode ser compreendida de forma intuitiva ao se interpretar as variáveis como **condições** e os operadores como **relações lógicas entre essas condições**.

$$1 + 1 = 1$$

Na lógica booleana, o operador **OU (OR)** indica a presença de pelo menos uma condição verdadeira. Quando se avalia **1 + 1**, ambas as entradas já representam condições verdadeiras. A repetição dessa condição não altera o estado lógico do sistema, pois uma condição verdadeira não se torna “**mais verdadeira**” por ocorrer duas vezes. Assim, o resultado permanece **1**.

$$1 \cdot 1 = 1$$

O operador **E (AND)** exige que todas as condições envolvidas sejam verdadeiras. Na expressão **1 · 1**, ambas as entradas satisfazem plenamente essa exigência. Dessa forma, o sistema reconhece que todas as condições necessárias estão presentes, produzindo como resultado o valor 1.

$$A + B = X$$

A expressão **A + B** representa uma situação em que a saída **X** depende da ocorrência de **A**, de **B**, ou de ambos. De maneira intuitiva, trata-se de uma condição alternativa, basta que uma das variáveis seja verdadeira para que o resultado seja

verdadeiro. O valor de **X** varia conforme os estados assumidos por **A** e **B**, sendo formalmente definido por sua tabela-verdade.

$$A + A = A$$

A operação **OR** aplicada entre uma variável e ela mesma representa a repetição de uma mesma condição lógica. Intuitivamente, verificar duas vezes a mesma condição não altera o resultado final. Se **A** é verdadeira, o resultado será verdadeiro, se **A** é falsa, o resultado será falso, dessa forma, **A + A** é logicamente equivalente a **A**.

$$A \cdot B = X$$

A expressão **A · B** descreve uma situação em que a saída **X** só será verdadeira se **A e B ocorrerem simultaneamente**. Intuitivamente, trata-se de uma condição conjunta, na qual todas as exigências precisam ser atendidas. Caso qualquer uma das variáveis seja falsa, o resultado final será **falso**.

$$A \cdot A = A$$

Assim como ocorre na operação **OR**, a aplicação do operador **AND** entre uma variável e ela mesma não modifica o estado lógico. Exigir que uma condição seja verdadeira duas vezes é logicamente equivalente a exigi-la apenas uma vez, portanto, **A · A** resulta em **A**.

$$A + 0 = A$$

Na operação **OR**, o valor **0** representa a ausência de uma condição verdadeira. Intuitivamente, adicionar uma condição que nunca ocorre não interfere no resultado final. Assim, o comportamento lógico do sistema depende exclusivamente do valor de **A**, resultando em **A + 0 = A**.

$$A \cdot 1 = A$$

Na operação AND, o valor 1 representa uma condição sempre satisfeita. Intuitivamente, exigir que uma condição verdadeira esteja presente não impõe nenhuma restrição adicional ao sistema. Dessa forma, o resultado lógico continua dependendo apenas de **A**, resultando em $A \cdot 1 = A$.

$$A + 1 = 1$$

Na operação OR, a presença de uma condição sempre verdadeira garante que o resultado será verdadeiro, independentemente de qualquer outra condição envolvida. Intuitivamente, se ao menos uma condição é garantidamente satisfeita, o sistema como um todo será verdadeiro.

$$A \cdot 0 = 0$$

Na operação AND, a presença de uma condição que nunca é satisfeita inviabiliza o resultado verdadeiro. Intuitivamente, se uma das exigências jamais ocorre, o conjunto de condições nunca poderá ser plenamente atendido, resultando sempre em 0.

$$A + A' = 1$$

A variável **A'** representa a negação lógica de **A** dado pela apóstrofe ('). Intuitivamente, uma condição e sua negação garantem que **uma delas sempre será verdadeira**, assim, a operação OR entre **A** e **A'** resulta inevitavelmente em 1.

$$A \cdot A' = 0$$

Por definição, uma condição e sua negação nunca podem ser verdadeiras simultaneamente. Intuitivamente, exigir que ambas ocorram ao mesmo tempo é logicamente impossível, dessa forma, a operação **AND** entre **A** e **A'** resulta sempre em 0.

A interpretação intuitiva desses termos evidencia que a álgebra de Boole não está associada ao cálculo numérico, mas à **avaliação de condições lógicas**. Essa perspectiva facilita a compreensão das propriedades algébricas e fundamenta as técnicas de **simplificação de expressões booleanas**, essenciais para a otimização de circuitos digitais e sistemas computacionais.

2.5 Construção de lógicas a partir de tabelas-verdade

Em muitos contextos da ciência da computação e dos sistemas digitais, a lógica de um sistema não é inicialmente descrita por meio de uma expressão algébrica, mas sim a partir de uma **tabela-verdade**. Essa abordagem consiste em especificar diretamente o valor da saída do sistema para **todas as combinações possíveis das variáveis de entrada**, garantindo uma descrição completa e inequívoca do comportamento lógico desejado.

A tabela-verdade é particularmente importante quando o problema a ser modelado envolve **regras, condições ou decisões bem definidas**, mas cuja expressão algébrica ainda não é evidente. Ao listar explicitamente todas as situações possíveis, o projetista consegue assegurar que nenhum caso foi negligenciado, evitando ambiguidades ou comportamentos inesperados no sistema final.

Tabela Verdade		
ENTRADAS		SAÍDA
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

Figura 5 – Construção de lógicas a partir desta tabela-verdade
Fonte: Elaborado pelo autor (2026)

No exemplo apresentado, a tabela-verdade define o valor da saída **X** para todas as combinações das entradas **A** e **B**. Observa-se que a saída assume o valor 1 quando **A e B possuem o mesmo valor lógico** e 0 quando assumem valores distintos. Esse tipo de lógica é comum em sistemas de comparação, validação e controle, nos quais o resultado depende diretamente da relação entre as entradas.

A construção de lógicas por meio de tabelas-verdade é amplamente utilizada em:

- Especificação de circuitos digitais;
- Projeto de sistemas embarcados;
- Definição de regras de decisão em algoritmos;
- Modelagem de funções lógicas em bancos de dados e sistemas de consulta.

Dessa forma, a tabela-verdade atua como uma **ferramenta fundamental de especificação**, permitindo que o comportamento lógico de um sistema seja definido de maneira clara, completa e independente de sua implementação física ou algébrica.

2.6 Extração de expressões booleanas a partir da tabela-verdade

Embora a tabela-verdade seja uma representação completa do comportamento de uma função booleana, ela nem sempre é a forma mais prática para análise, implementação ou manipulação lógica. Por esse motivo, é comum **extrair uma**

expressão booleana equivalente a partir da tabela-verdade, permitindo que o comportamento do sistema seja tratado de forma algébrica.

Tabela Verdade		
ENTRADAS		SAÍDA
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

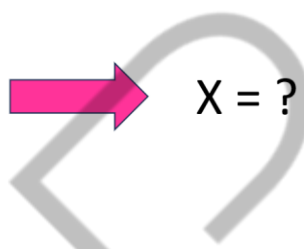


Figura 6 – Expressão lógica da tabela-verdade a ser extraída
Fonte: Elaborado pelo autor (2026)

A partir das linhas da tabela em que a saída assume o valor 1 (ou, alternativamente, o valor 0), é possível construir uma expressão lógica que reproduza exatamente o mesmo comportamento da função. Essa expressão representa, de forma simbólica, todas as condições sob as quais a saída do sistema é verdadeira ou falsa.

Entretanto, as expressões booleanas obtidas diretamente da tabela-verdade tendem a ser **longas e redundantes**, pois refletem fielmente cada combinação individual das variáveis de entrada. Essa redundância não compromete a correção lógica da expressão, mas pode resultar em representações excessivamente complexas e pouco eficientes para implementação em circuitos digitais ou algoritmos.

Assim, a extração da expressão booleana constitui um **passo intermediário fundamental** entre a especificação completa do comportamento lógico (tabela-verdade) e a otimização desse comportamento por meio de técnicas algébricas. Nesse estágio, o objetivo não é ainda reduzir ou simplificar a expressão, mas apenas **traduzir a lógica tabular para uma forma algébrica equivalente**, que posteriormente poderá ser analisada e aprimorada.

Esse processo reforça a ideia de que diferentes representações de tabelas-verdade e expressões booleanas, descrevem o mesmo comportamento lógico, cada uma com vantagens específicas conforme o contexto de uso. Nas próximas seções,

essas expressões serão utilizadas como base para o estudo das técnicas de simplificação algébrica.

2.7 Mintermos e maxtermos, extração de expressões booleanas a partir da tabela-verdade

No estudo da lógica booleana, a **tabela-verdade** representa a forma mais completa e explícita de descrever o comportamento de uma função booleana, pois enumera todas as combinações possíveis das variáveis de entrada e o valor correspondente da saída. A partir dessa tabela, é possível extrair expressões algébricas padronizadas conhecidas como **mintermos** e **maxtermos**, que desempenham papel fundamental na análise e na simplificação de funções lógicas.

2.7.1 Mintermos

Um **mintermo** é um termo booleano que representa **uma única linha da tabela-verdade em que a saída da função assume o valor (1)**. Cada mintermo corresponde a uma combinação específica das variáveis de entrada, na qual todas elas aparecem exatamente uma vez, seja em sua forma direta ou complementada.

De forma intuitiva, pode-se afirmar que um mintermo descreve uma **condição mínima suficiente** para que a saída da função seja verdadeira. Para construir um mintermo:

- Cada variável que assume valor **1** na linha da tabela aparece em sua forma direta;
- Cada variável que assume valor **0** aparece em sua forma complementada.

Os mintermos são conectados entre si pela operação lógica **OU (OR)**, formando a chamada **forma canônica soma de mintermos**, que garante que a função será verdadeira sempre que qualquer um desses mintermos for satisfeito.

Do ponto de vista conceitual, os mintermos representam **todos os cenários em que o sistema deve produzir saída verdadeira**, sendo amplamente utilizados na descrição formal de funções booleanas e na síntese de circuitos digitais.

2.7.2 Maxtermos

Um **maxtermo**, por sua vez, é um termo booleano que representa **uma única linha da tabela-verdade em que a saída da função assume o valor 0**. Assim como os mintermos, os maxtermos também incluem todas as variáveis da função, cada uma aparecendo exatamente uma vez, direta ou complementada.

Intuitivamente, um maxtermo descreve uma **condição máxima que impede** a saída de ser verdadeira. Para construir um maxtermo:

- Cada variável que assume valor **0** na linha da tabela aparece em sua forma direta;
- Cada variável que assume valor **1** aparece em sua forma complementada.

Os maxtermos são conectados entre si pela operação lógica **E (AND)**, originando a **forma canônica produto de maxtermos**. Essa forma garante que a função será falsa sempre que qualquer um dos maxtermos for satisfeito.

2.7.3 Relação entre mintermos, maxtermos e a tabela-verdade

A principal diferença conceitual entre mintermos e maxtermos está no **foco da representação**:

- Os **mintermos** enfatizam os casos em que a saída é **1**;
- Os **maxtermos** enfatizam os casos em que a saída é **0**.

MINTERMOS E MAXTERMOS

Mintermos

ENTRADAS		SAÍDA
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

→ $A' + B'$

→ $A + B$

Maxtermos

ENTRADAS		SAÍDA
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

→ $A + B'$

→ $A' + B$

Figura 7 – Comparação dos termos extraídos pelos mintermos e maxtermos
Fonte: Elaborado pelo autor (2026)

Ambas as representações são **equivalentes do ponto de vista funcional**, pois descrevem a mesma função booleana, apenas sob perspectivas complementares. Toda função booleana pode ser representada integralmente tanto como soma de mintermos quanto como produto de maxtermos.

2.8 Importância acadêmica e prática

O estudo de mintermos e maxtermos é fundamental porque:

- Estabelece uma ligação direta entre a tabela-verdade e a expressão algébrica;
- Fornece uma representação padronizada e completa das funções booleanas;
- Serve de base para técnicas de simplificação, como mapas de Karnaugh e métodos algébricos;
- Facilita a implementação sistemática de circuitos digitais a partir de especificações lógicas.

Do ponto de vista formativo, compreender mintermos e maxtermos permite ao estudante perceber que qualquer comportamento lógico pode ser rigorosamente descrito a partir de dados tabulares, reforçando a ideia central da ciência da

computação de que sistemas complexos podem ser modelados formalmente por meio de estruturas matemáticas bem definidas.

2.9 Extração da expressão booleana por mintermos (forma soma de produtos)

Em muitos casos, após a definição do comportamento lógico de um sistema por meio de uma **tabela-verdade**, torna-se necessário converter essa representação tabular em uma **expressão booleana equivalente**. Um dos métodos mais sistemáticos e conceitualmente importantes para essa conversão é a **extração da expressão por meio de mintermos**.

2.9.1 Identificação dos mintermos a partir da saída igual a 1

O princípio fundamental da extração por mintermos é que **cada mintermo corresponde a uma linha da tabela-verdade em que a saída da função assume o valor 1**. Em outras palavras, os mintermos descrevem **todas as condições sob as quais o sistema deve produzir saída verdadeira**.

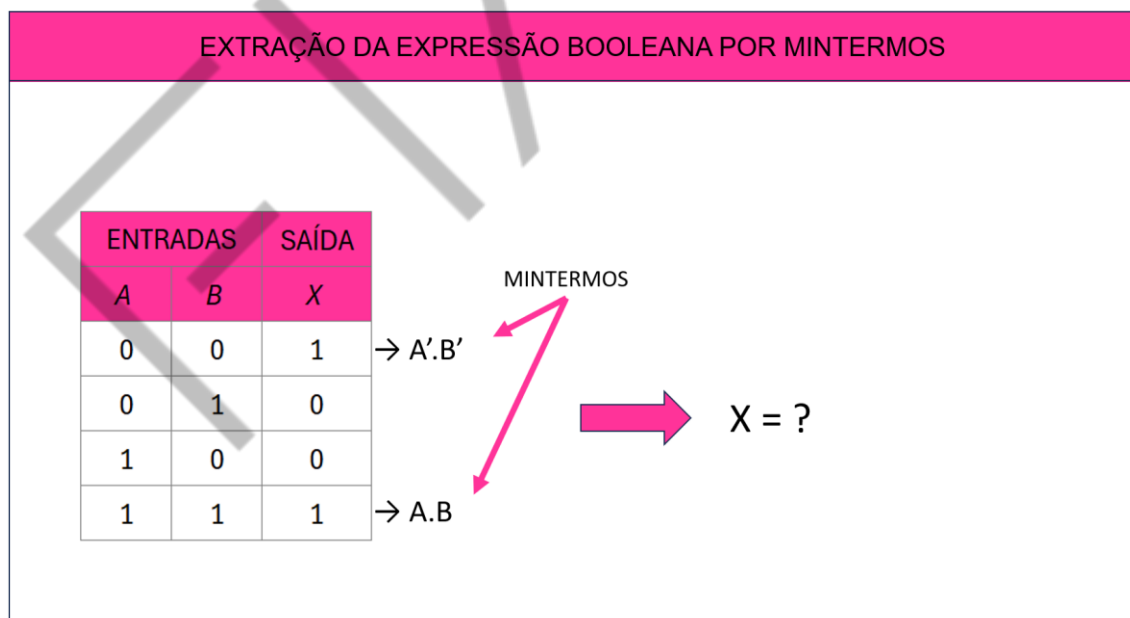


Figura 8 – Identificando os mintermos desta tabela-verdade
Fonte: Elaborado pelo autor (2026)

No exemplo apresentado, observa-se que a saída $X = 1$ ocorre nas seguintes combinações das entradas:

- $A = 0, B = 0$

- $A = 1, B = 1$

Cada uma dessas linhas dará origem a um mintermo distinto, pois cada uma representa um cenário específico no qual a função deve ser verdadeira.

2.9.2 Por que negar variáveis quando a entrada é 0

Ao construir um mintermo, todas as variáveis da função devem aparecer **explicitamente**, seja em sua forma direta ou complementada. A regra é a seguinte:

- Se a variável assume valor **1** na linha da tabela, ela aparece **sem negação**;
- Se a variável assume valor **0**, ela aparece **negada**.

ENTRADAS		SAÍDA	
A	B	X	
0	0	1	$\rightarrow A'.B'$
0	1	0	
1	0	0	
1	1	1	$\rightarrow A.B$

(') Representa a Negação das entradas
 $A=0$ e $B=0 \rightarrow A'$ e B'
 $A=1$ e $B=1 \rightarrow A$ e B

Figura 9 – Identificando os mintermos desta tabela-verdade e negando os zeros
 Fonte: Elaborado pelo autor (2026)

Essa escolha garante que o mintermo represente **exatamente aquela combinação específica de entradas**, e nenhuma outra.

No exemplo:

- Para $A = 0$ e $B = 0$, o mintermo é $A' \cdot B'$.
- Para $A = 1$ e $B = 1$, o mintermo é $A \cdot B$.

A negação das variáveis que assumem valor 0 é necessária porque o mintermo deve ser verdadeiro **somente naquela linha da tabela**, e falso em todas as demais.

2.9.3 Uso do operador AND na construção de cada mintermo

Dentro de cada mintermo, as variáveis são conectadas pelo operador lógico **E (AND)**. Isso ocorre porque o mintermo representa uma **condição simultânea**, ou seja, todas as variáveis devem assumir exatamente os valores especificados para que aquele termo seja verdadeiro.

Por exemplo, o mintermo $A' \cdot B'$ só será igual a 1 quando $A = 0$ e $B = 0$ **ao mesmo tempo**. Qualquer variação nessas entradas faz com que o termo seja avaliado como 0.

Assim, o operador **AND** é utilizado porque o mintermo descreve uma **combinação única e específica de entradas**.

2.9.4 Formação da expressão final com o operador OR

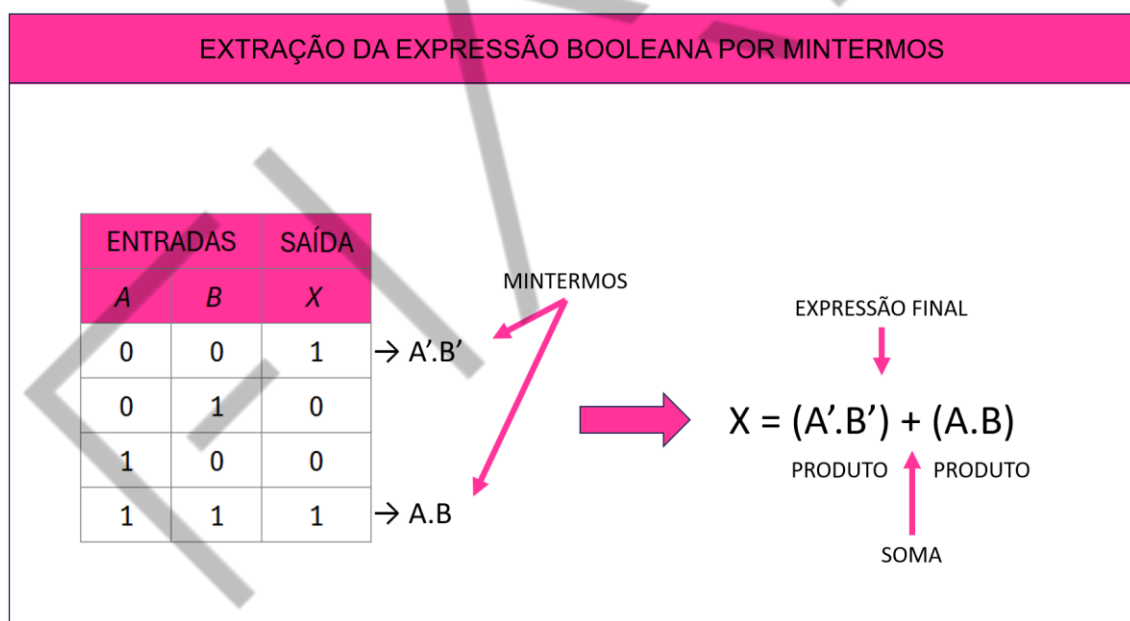


Figura 10 – Extração da expressão pelos mintermos, soma de produtos
Fonte: Elaborado pelo autor (2026)

Após a construção de todos os mintermos correspondentes às linhas em que $X = 1$, a expressão booleana final é obtida pela **soma lógica (OR)** de todos esses termos.

No exemplo analisado, a expressão resultante é:

$$X = (A' \cdot B') + (A \cdot B)$$

O uso do operador **OR** indica que a saída será verdadeira **sempre que qualquer um dos mintermos for satisfeito**. Ou seja, basta que o sistema esteja em **uma das combinações listadas** para que a saída seja igual a 1

2.9.5 Significado da forma “soma de produtos”

O diagrama apresenta a expressão matemática $X = (A'.B') + (A.B)$ no centro. Acima dela, um retângulo rosa contém o texto 'SOMA DE PRODUTOS'. Abaixo da expressão, há três setas verticais cor-de-rosa apontando para cima. Duas dessas setas, uma sob $A'.B'$ e outra sob $A.B$, são rotuladas com o texto 'PRODUTO'. A terceira seta, localizada sob o símbolo de adição '+', é rotulada com o texto 'SOMA'.

$$X = (A'.B') + (A.B)$$

PRODUTO PRODUTO

SOMA

Figura 11 – Forma canônica “Soma de Produtos”
Fonte: Elaborado pelo autor (2025)

A expressão obtida por meio de mintermos é conhecida como **forma canônica “Soma de Produtos” (Sum of Products – SOP)**. Esse nome decorre diretamente de sua estrutura:

- Cada **produto** corresponde a um mintermo (variáveis combinadas por **AND**);
- A **soma** corresponde à união lógica (**OR**) de todos os mintermos.

Essa forma é chamada de **canônica** porque segue um padrão rigoroso e completo, no qual todas as variáveis aparecem em todos os termos. Embora essa representação nem sempre seja a mais eficiente do ponto de vista de implementação, ela possui grande importância conceitual, pois garante uma correspondência direta e inequívoca entre a **tabela-verdade** e a **expressão algébrica**.

A extração por mintermos constitui, portanto, um passo fundamental no estudo da lógica booleana, servindo como base para técnicas posteriores de **simplificação algébrica** e **otimização de circuitos**, que serão abordadas nas próximas seções.

2.10 Extração da expressão booleana por meio de maxtermos (forma produto de somas)

De maneira complementar à extração por mintermos, a lógica booleana permite obter uma expressão algébrica equivalente a partir dos **maxtermos**. Esse método também parte da tabela-verdade, porém adota uma **perspectiva oposta**, concentrando-se nas linhas em que a saída da função assume o valor **0**.

2.10.1 Identificação dos maxtermos a partir da saída igual a 0

O princípio central da extração por maxtermos é que **cada maxtermo corresponde a uma linha da tabela-verdade em que a saída da função é igual a 0**. Essas linhas representam **as condições sob as quais o sistema não deve produzir saída verdadeira**.

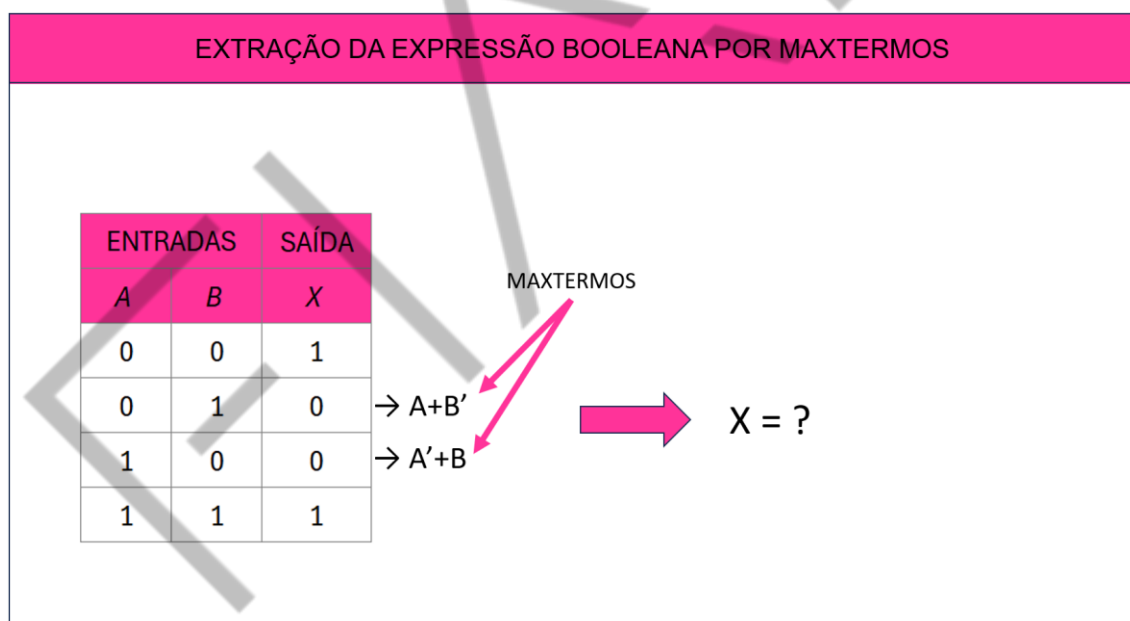


Figura 12 – Extração da expressão booleana por maxtermos
Fonte: Elaborado pelo autor (2026)

No exemplo apresentado, observa-se que a saída $X = 0$ ocorre nas seguintes combinações das entradas:

- $A = 0, B = 1$
- $A = 1, B = 0$

Cada uma dessas linhas dará origem a um maxtermo distinto, pois cada uma representa uma condição específica que força a saída a ser falsa.

2.10.2 Por que as variáveis não são negadas quando a entrada é 0?

A construção de um maxtermo segue uma lógica diferente daquela utilizada nos mintermos. As regras são:

- Se a variável assume valor **0** na linha da tabela, ela aparece **sem negação**;
- Se a variável assume valor **1**, ela aparece **negada**.

ENTRADAS		SAÍDA	
A	B	X	
0	0	1	
0	1	0	$\rightarrow A+B'$
1	0	0	$\rightarrow A'+B$
1	1	1	

(') Representa a Negação das entradas neste caso quando são 1s.

$A=0$ e $B=1 \rightarrow A$ e B'

$A=1$ e $B=0 \rightarrow A'$ e B

Figura 13 – Extração dos termos por maxtermos, negando os 1s
Fonte: Elaborado pelo autor (2026)

Essa convenção garante que o maxtermo será avaliado como **0 exatamente naquela combinação de entradas** e como **1 em todas as demais**.

No exemplo:

- Para $A = 0$ e $B = 1$, o maxtermo é $(A + B')$;
- Para $A = 1$ e $B = 0$, o maxtermo é $(A' + B)$.

Observe que, em cada maxtermo, apenas uma das variáveis precisa assumir um valor específico para que o termo seja satisfeito, característica fundamental da operação OR.

2.10.3 Uso do operador OR na construção de cada maxtermo

Dentro de cada maxtermo, as variáveis são combinadas pelo operador lógico **OU (OR)**. Isso ocorre porque o maxtermo descreve uma **condição alternativa**, basta que uma das variáveis esteja em determinado estado para que o termo seja avaliado como verdadeiro.

Entretanto, o maxtermo foi construído de forma que ele se torne **falso apenas na linha da tabela que representa**, garantindo a correspondência exata com a condição $X = 0$.

2.10.4 Formação da expressão final com o operador AND

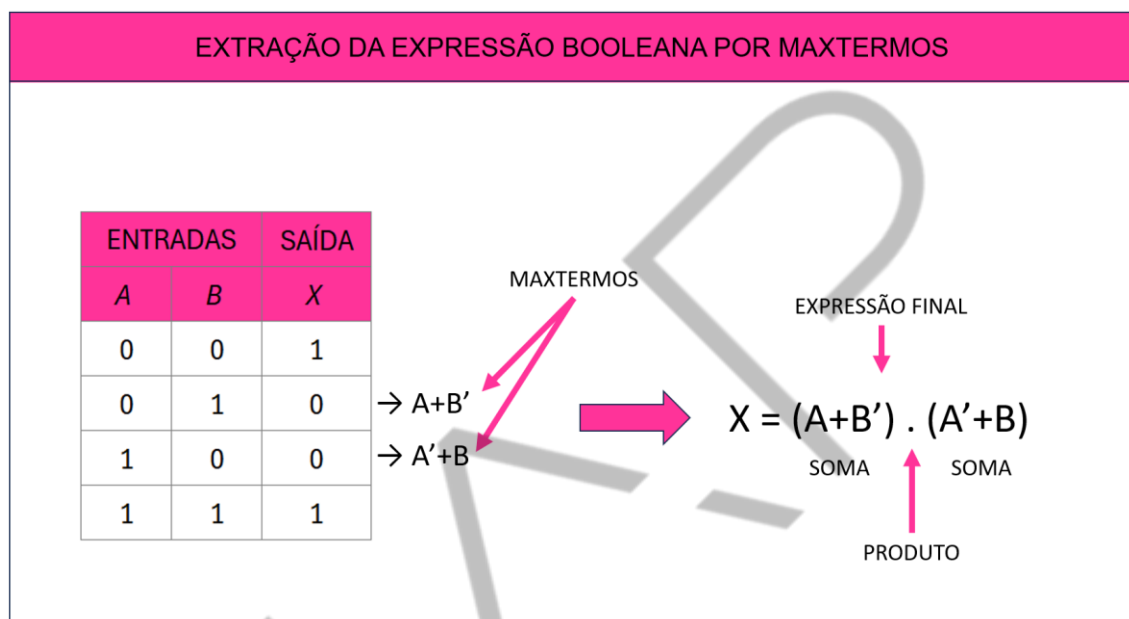


Figura 14 – Extração da expressão pelos maxtermos, produto de somas
Fonte: Elaborado pelo autor (2026)

Após a construção de todos os maxtermos correspondentes às linhas em que $X = 0$, a expressão booleana final é obtida pelo **produto lógico (AND)** entre todos esses termos.

No exemplo analisado, a expressão resultante é:

$$X = (A + B') \cdot (A' + B)$$

O uso do operador AND indica que a saída será verdadeira **somente se todos os maxtermos forem satisfeitos simultaneamente**, isto é, desde que o sistema **não** esteja em nenhuma das condições que produzem saída 0.

2.10.5 Significado da forma “produto de somas”

O diagrama mostra a expressão matemática $X = (A+B') \cdot (A'+B)$. Acima da expressão, há um retângulo magenta com o texto "PRODUTO DE SOMAS". Abaixo da expressão, há três setas magenta apontando para cima. Duas setas apontam para os parênteses $(A+B')$ e $(A'+B)$, cada uma com o rótulo "SOMA" abaixo. Uma seta centralizada aponta para o operador de multiplicação $\cdot$, com o rótulo "PRODUTO" abaixo.

$$X = (A+B') \cdot (A'+B)$$

SOMA SOMA

PRODUTO

Figura 15 – Forma canônica “Produto de Somas”
Fonte: Elaborado pelo autor (2026)

A expressão obtida por meio de maxtermos é conhecida como **forma canônica “Produto de Somas” (Product of Sums – POS)**. Essa denominação decorre diretamente de sua estrutura:

- Cada **soma** corresponde a um maxtermo (variáveis combinadas por **OR**);
- O **produto** corresponde à combinação lógica (**AND**) de todos os maxtermos.

Assim como ocorre com a soma de produtos, a forma produto de somas é uma representação **completa e padronizada** da função booleana. Embora nem sempre seja a forma mais eficiente para implementação direta, ela é extremamente importante do ponto de vista conceitual, pois estabelece uma correspondência rigorosa entre a **tabela-verdade** e a **expressão algébrica**.

2.11 Equivalência lógica entre mintermos e maxtermos

As formas canônicas **soma de produtos (mintermos)** e **produto de somas (maxtermos)** descrevem a **mesma função booleana**, embora partam de perspectivas distintas da tabela-verdade. Enquanto os **mintermos** são construídos a partir das linhas em que a saída é **1**, os **maxtermos** são construídos a partir das linhas em que a saída é **0**. Apesar dessa diferença conceitual, **ambas produzem exatamente os mesmos valores de saída para todas as combinações de entrada**.

Considere as duas expressões a seguir, obtidas para a mesma função lógica:

- Forma soma de produtos (mintermos):

$$X = (A' \cdot B') + (A \cdot B)$$

- Forma produto de somas (maxtermos):

$$X = (A + B') \cdot (A' + B)$$

Embora apresentem estruturas algébricas diferentes, elas são **logicamente equivalentes**. Isso significa que, para qualquer valor de **A** e **B**, ambas produzem o **mesmo valor de X**.

2.11.1 Verificação por meio da tabela-verdade

A equivalência entre as duas expressões pode ser comprovada avaliando-se seus resultados para todas as combinações possíveis das entradas:

EQUIVALÊNCIA LÓGICA ENTRE MINTERMOS E MAXTERMOS

MINTERMOS

$$X = (A'.B') + (A.B)$$

ENTRADAS		SAÍDA
A	B	$X=(A'.B')+(A.B)$
0	0	1
0	1	0
1	0	0
1	1	1

MAXTERMOS

$$X = (A+B') . (A'+B)$$

ENTRADAS		SAÍDA
A	B	$X=(A+B') \cdot (A'+B)$
0	0	1
0	1	0
1	0	0
1	1	1

Figura 16 – Equivalência lógica entre mintermos e maxtermos
Fonte: Elaborado pelo autor (2026)

Observa-se que, para todas as combinações de **A** e **B**, o valor final de **X** é o mesmo nas duas formas. Assim, embora uma expressão tenha sido construída a partir dos **casos verdadeiros** (mintermos) e a outra a partir dos **casos falsos** (maxtermos), **o comportamento lógico do sistema permanece inalterado**.

2.11.2 Interpretação conceitual

Essa equivalência revela um ponto fundamental da lógica booleana:

- Os **mintermos** explicitam **quando** a saída deve ser verdadeira;
- Os **maxtermos** explicitam **quando** a saída não deve ser verdadeira.

Ambas as descrições são completas e rigorosas, apenas enfatizando aspectos opostos do mesmo comportamento lógico. Por isso, qualquer função booleana pode ser representada **integralmente** em ambas as formas.

O domínio dessas duas representações prepara o estudante para compreender, nas seções seguintes, como expressões inicialmente **extensas e redundantes** podem ser transformadas em formas **mais simples e eficientes**, sem alterar em nenhum momento os valores de saída da função lógica.

3 SIMPLIFICAÇÃO ALGÉBRICA

A **simplificação algébrica** consiste no processo de transformar uma expressão booleana em uma **forma equivalente mais simples**, preservando integralmente o seu comportamento lógico. Duas expressões são consideradas equivalentes quando, para todas as combinações possíveis de suas variáveis de entrada, produzem **exatamente os mesmos valores de saída**, como visto na figura a seguir.

FORMA EQUIVALENTE MAIS SIMPLES					
EXPRESSÃO SEM SIMPLIFICAÇÃO			EXPRESSÃO SIMPLIFICADA		
$X = A + A + (A \cdot B)$			$X = A$		
ENTRADAS		SAÍDA	ENTRADAS		SAÍDA
A	B	$X = A + A + (A \cdot B)$	A	B	$X = A$
0	0	0	0	0	0
0	1	0	0	1	0
1	0	1	1	0	1
1	1	1	1	1	1

Figura 17 – Forma equivalente mais simples
 Fonte: Elaborado pelo autor (2026)

As expressões booleanas extraídas diretamente de **tabelas-verdade**, seja pelo método dos **mintermos** ou dos **maxtermos**, tendem a apresentar uma estrutura **extensa e redundante**. Isso ocorre porque esses métodos descrevem o comportamento lógico de forma completa e explícita, representando individualmente cada combinação relevante de entradas. Embora corretas do ponto de vista funcional, tais expressões nem sempre são adequadas para implementação prática, análise ou manutenção.

Entretanto, a necessidade de simplificação algébrica **não se restringe às expressões obtidas a partir de tabelas-verdade**. Em muitos casos, expressões booleanas surgem naturalmente durante o processo de **construção de lógicas de programação** e de **lógicas digitais**, como resultado de decisões encadeadas, condições aninhadas ou combinações sucessivas de regras lógicas. Nessas situações, a expressão construída pode ser funcionalmente correta, mas estruturalmente mais complexa do que o necessário.

No contexto da **programação**, expressões booleanas aparecem com frequência em estruturas condicionais, como comandos *if*, *while*, *for* e operadores lógicos compostos. Expressões excessivamente longas ou redundantes podem dificultar a leitura do código, aumentar a probabilidade de erros e comprometer a clareza da lógica implementada. A simplificação algébrica contribui, nesse cenário,

para a produção de código **mais legível, mais conciso e mais fácil de manter**, sem alterar o comportamento do programa.

Já no contexto da **lógica digital**, a simplificação algébrica possui implicações diretas na implementação física dos sistemas. Cada termo e cada operação lógica adicional geralmente correspondem a **mais portas lógicas, mais conexões e maior complexidade do circuito**. A redução de expressões booleanas permite diminuir o número de componentes necessários, reduzir o consumo de energia, aumentar a confiabilidade do sistema e simplificar os processos de projeto, montagem e teste.

Do ponto de vista teórico, a simplificação algébrica baseia-se nas **leis e propriedades da álgebra de Boole**, que permitem reorganizar, combinar e eliminar termos redundantes sem modificar o valor lógico da expressão. Esse processo não altera a função representada, mas apenas sua **forma algébrica**, tornando-a mais eficiente e elegante.

Assim, a simplificação algébrica deve ser compreendida como uma **etapa fundamental do raciocínio lógico em ciência da computação**, aplicável tanto à análise formal de funções booleanas quanto ao desenvolvimento prático de software e hardware. Nas seções seguintes, serão apresentadas as principais técnicas e leis utilizadas nesse processo, sempre com o objetivo de reduzir a complexidade das expressões mantendo-se inalterado o comportamento lógico do sistema.

3.1 Simplificação algébrica com eliminação de redundâncias e reorganização de termos

Considere a seguinte expressão booleana inicial:

$$X = (A + B + A + B) \cdot A$$

Essa expressão apresenta **redundâncias explícitas**, tanto na repetição de variáveis quanto na forma de organização dos termos. A simplificação algébrica permite eliminar essas redundâncias de maneira sistemática, preservando o comportamento lógico da função.

3.1.1 Reorganização dos termos (lei comutativa)

Inicialmente, aplica-se a **lei comutativa da operação OR**, que permite reorganizar os termos sem alterar o resultado lógico:

$$X = (A + A + B + B) \cdot A$$

Essa etapa não altera o valor da expressão, mas facilita a identificação de termos repetidos.

3.1.2 Eliminação de termos repetidos (lei da idempotência)

Na álgebra de Boole, a repetição de uma mesma variável na operação **OR** não altera o resultado:

$$A + A = A$$

e

$$B + B = B$$

$$X = (\cancel{A} + \cancel{A} + B + B) \cdot A$$
$$X = (A + B) \cdot A$$

Figura 18 – Eliminação de termos repetidos (lei da idempotência)
Fonte: Elaborado pelo autor (2026)

Aplicando essa propriedade, a expressão se reduz a:

$$X = (A + B) \cdot A$$

Essa etapa evidencia que múltiplas ocorrências da mesma variável não aumentam a “força” lógica da condição.

3.1.3 Aplicação da lei distributiva

Em seguida, aplica-se a **lei distributiva**, expandindo o produto lógico:

$$X = A \cdot A + A \cdot B$$

$$X = (A + B) \cdot A$$
$$X = (A \cdot A + A \cdot B)$$

Figura 19 – Aplicação da lei distributiva
Fonte: Elaborado pelo autor (2026)

3.1.4 Nova aplicação da lei da idempotência

Utiliza-se novamente a propriedade:

$$A \cdot A = A$$

Resultando em:

$$X = A + A \cdot B$$

3.1.5 Fatoração da variável comum

Observa-se que a variável **A** é comum aos dois termos da expressão. A **fatoração** consiste em colocá-la em evidência:

$$X = A(1 + B)$$

$$X = A + A \cdot B$$
$$X = A \cdot (1 + 1 \cdot B)$$
$$X = A \cdot (1 + B)$$

Figura 20 – Fatoração da variável comum
Fonte: Elaborado pelo autor (2026)

A fatoração é uma etapa conceitualmente importante, pois permite revelar constantes e relações lógicas que não são evidentes na forma expandida.

3.1.6 Aplicação da lei da dominação

Na lógica booleana, a operação OR com o valor 1 resulta sempre em 1:

$$1 + B = 1$$

Substituindo na expressão:

$$X = A \cdot (1)$$

3.1.7 Aplicação da lei da identidade

Por fim, aplica-se a propriedade:

$$A \cdot 1 = A$$

Obtendo-se a forma final simplificada:

$$\boxed{X = A}$$

3.1.8 Interpretação conceitual da simplificação

Esse exemplo demonstra como expressões booleanas aparentemente complexas podem ser reduzidas a formas extremamente simples quando as **leis da álgebra de Boole** são aplicadas de maneira sistemática. A expressão original continha múltiplas repetições e operações desnecessárias, que não contribuíam para o comportamento lógico final do sistema.

Do ponto de vista da **lógica digital**, essa simplificação implica a eliminação de portas lógicas redundantes, resultando em um circuito mais simples, econômico e confiável. No contexto da **programação**, o mesmo raciocínio corresponde à remoção de condições repetidas ou irrelevantes, tornando o código mais claro e eficiente.

Esse tipo de análise reforça a importância da simplificação algébrica como uma ferramenta essencial para transformar soluções corretas em soluções **otimizadas**, mantendo-se sempre inalterado o comportamento lógico da função.

3.2 Exemplo de simplificação algébrica utilizando as leis da álgebra de Boole

Considere a seguinte expressão booleana:

$$X = (A + B) \cdot A + A$$

Essa expressão é funcionalmente correta, porém apresenta **redundâncias lógicas** que podem ser eliminadas por meio da **simplificação algébrica**. O objetivo desse processo é obter uma expressão **mais simples**, mantendo exatamente o mesmo comportamento lógico para todas as combinações das variáveis, como observado na figura a seguir.

$$\begin{aligned} X &= (A + B) \cdot A + A \\ X &= A \cdot A + A \cdot B + A && \text{Lei Distributiva} \\ X &= A + A \cdot B + A && A \cdot A = A \\ X &= A + A + A \cdot B && \text{Reorganizar para melhor visualização} \\ X &= A + A \cdot B && \text{Fatoração da Variável, A em evidencia} \\ X &= A \cdot (1 + B) && 1 + B = B \\ X &= A \cdot (1) && A \cdot 1 = A \\ X &= A \end{aligned}$$

Figura 21 – Simplificação algébrica utilizando as leis da álgebra de Boole
Fonte: Elaborado pelo autor (2026)

Apresentaremos, de forma detalhada e justificativa, o passo a passo do processo de simplificação desta expressão.

3.2.1 Aplicação da lei distributiva

O primeiro passo consiste em aplicar a **lei distributiva da álgebra de Boole**, que permite distribuir o operador **AND** sobre o operador **OR**, de forma análoga à álgebra tradicional:

$$(A + B) \cdot A = A \cdot A + B \cdot A$$

Substituindo na expressão original, tem-se:

$$X = A \cdot A + B \cdot A + A$$

3.2.2 Aplicação da lei distributiva

A álgebra de Boole estabelece que a multiplicação lógica de uma variável por ela mesma não altera seu valor:

$$A \cdot A = A$$

Aplicando essa propriedade, a expressão torna-se:

$$X = A + B \cdot A + A$$

Observa-se agora que o termo **A** aparece repetido.

3.2.3 Eliminação de redundância (lei da idempotência no OR)

Na operação **OR**, a repetição de uma mesma variável não altera o resultado lógico:

$$A + A = A$$

Assim, a expressão pode ser reescrita como:

$$X = A + B \cdot A$$

3.2.4 Fatoração da variável comum

Nesse ponto, nota-se que a variável **A** é comum aos dois termos da expressão. A **fatoração** consiste em colocar essa variável em evidência, facilitando a aplicação de outras propriedades algébricas:

$$X = A(1 + B)$$

Essa etapa é importante porque **revela estruturas lógicas ocultas**, permitindo identificar constantes dominantes e simplificações adicionais.

3.2.5 Justificativa de que $(1 + B = 1)$

Na lógica booleana, a operação **OR** com o valor **1** sempre resulta em **1**, independentemente do valor da outra variável:

$$1 + B = 1$$

Isso ocorre porque o valor **1** representa uma condição sempre verdadeira, que domina a operação **OR**.

Substituindo na expressão:

$$X = A(1)$$

3.2.6 Justificativa de que $(A \cdot 1 = A)$

A operação **AND** com o valor **1** não altera o valor lógico da variável:

$$A \cdot 1 = A$$

O valor **1** funciona como o **elemento neutro** da operação **AND**.

Assim, chega-se à forma final simplificada:

$$\boxed{X = A}$$

3.2.7 Interpretação conceitual da simplificação

Esse exemplo ilustra claramente por que a simplificação algébrica é uma etapa fundamental na lógica booleana:

- A expressão original continha termos redundantes;
- A aplicação sistemática das leis algébricas permitiu eliminar operações desnecessárias;
- O comportamento lógico final foi preservado integralmente.

Do ponto de vista da **lógica digital**, essa simplificação significa reduzir o circuito a **uma única ligação direta da entrada A para a saída X**, eliminando portas lógicas desnecessárias. No contexto da **programação**, equivale a remover condições redundantes, tornando o código mais simples, legível e eficiente. Esse tipo de raciocínio constitui a base para todas as técnicas formais de **simplificação algébrica**.

3.3 Simplificação algébrica por transformações equivalentes e fatoração sucessiva

Considere a expressão booleana inicial:

$$X = A + (B \cdot C) + A + C$$

Esse exemplo sintetiza diversas **técnicas fundamentais de simplificação algébrica**, mostrando como expressões aparentemente extensas podem ser reduzidas de forma sistemática por meio da aplicação encadeada das leis da álgebra de Boole, como observado na seguinte figura.

$$\begin{array}{l}
 \boxed{X = A + (B \cdot C) + A + C} \\
 A + A = A \rightarrow X = \cancel{A} + (B \cdot C) + \cancel{A} + C \\
 X = A + (B \cdot C) + C \leftarrow \text{Transformação algébrica equivalente} \\
 \text{Lei Distributiva} \rightarrow X = (A + B) \cdot (A + C) + C \\
 A \cdot A = A \rightarrow X = A \cdot A + A \cdot C + A \cdot B + B \cdot C + C \\
 X = \cancel{A} + \cancel{A} \cdot C + A \cdot B + B \cdot C + C \leftarrow \text{Fatoração da Variável, A em evidência} \\
 \text{Qualquer variável} + 1 = 1 \rightarrow X = A \cdot (1 + 1 \cdot C) + A \cdot B + B \cdot C + C \\
 A \cdot 1 = A \rightarrow X = A \cdot (\cancel{1}) + A \cdot B + B \cdot C + C \\
 X = A + A \cdot B + B \cdot C + C \leftarrow \text{Fatoração da Variável, C em evidência} \\
 \text{Qualquer variável} + 1 = 1 \rightarrow X = A + A \cdot B + C \cdot (B \cdot 1 + 1) \\
 C \cdot 1 = C \rightarrow X = A + A \cdot B + C \cdot (\cancel{1}) \\
 X = A + A \cdot B + C \leftarrow \text{Fatoração da Variável, A em evidência} \\
 \text{Qualquer variável} + 1 = 1 \rightarrow X = A \cdot (1 + 1 \cdot B) + C \\
 X = A \cdot (\cancel{1}) + C \leftarrow A \cdot 1 = A \\
 \text{Expressão Simplifica} \rightarrow X = A + C
 \end{array}$$

Figura 22 – Simplificação algébrica por transformações equivalentes e fatoração sucessiva
Fonte: Elaborado pelo autor (2026)

Apresentaremos, de forma detalhada e justificativa, o passo a passo do processo de simplificação desta expressão.

3.3.1 Eliminação de termos repetidos (lei da idempotência)

Inicialmente, observa-se a repetição do termo **A** na expressão. Pela **lei da idempotência**, a soma lógica de uma variável com ela mesma não altera o resultado:

$$A + A = A$$

Assim, a expressão pode ser reescrita como:

$$X = A + (B \cdot C) + C$$

Essa etapa já reduz a complexidade da expressão, eliminando redundâncias explícitas.

3.3.2 Transformação algébrica equivalente

Em seguida, aplica-se uma **transformação algébrica equivalente** ao termo:

$$A + (B \cdot C)$$

Utilizando a distributiva da álgebra de Boole, esse termo pode ser reescrito como:

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Substituindo na expressão original, obtém-se:

$$X = (A + B) \cdot (A + C) + C$$

Essa transformação não altera o comportamento lógico da função, mas cria uma estrutura favorável à aplicação de novas simplificações.

3.3.3 Aplicação da lei distributiva

Expandindo o produto lógico, tem-se:

$$X = A \cdot A + A \cdot C + A \cdot B + B \cdot C + C$$

Aplicando a **lei da idempotência**:

$$A \cdot A = A$$

Resulta em:

$$X = A + A \cdot C + A \cdot B + B \cdot C + C$$

3.3.4 Fatoração sucessiva de termos comuns

Observa-se inicialmente que o termo **A** pode ser colocado em evidência:

$$X = A(1 + C) + A \cdot B + B \cdot C + C$$

Como **1 + C = 1**, pela **lei da dominação**, tem-se:

$$X = A + A \cdot B + B \cdot C + C$$

Em seguida, nota-se que o termo **C** também pode ser fatorado:

$$X = A + A \cdot B + C(B + 1)$$

Como $B + 1 = 1$, obtém-se:

$$X = A + A \cdot B + C$$

3.3.5 Fatoração final e aplicação das leis da álgebra de Boole

Por fim, observa-se novamente a presença do termo **A** em dois componentes da expressão:

$$X = A + A \cdot B + C$$

Colocando **A** em evidência:

$$X = A(1 + B) + C$$

Como $1 + B = 1$, resulta:

$$X = A + C$$

3.3.6 Resultado final da simplificação

$$\boxed{X = A + C}$$

3.3.7 Interpretação conceitual do exemplo

Esse exemplo final demonstra como a **simplificação algébrica pode exigir múltiplas etapas**, combinando:

- Eliminação de redundâncias;
- Transformações algébricas equivalentes;
- Aplicação da distributiva;
- Fatorações sucessivas;

- Uso sistemático das leis da álgebra de Boole.

Embora o processo intermediário envolva expressões aparentemente mais longas, cada etapa é logicamente equivalente à anterior. O resultado final, extremamente simples, evidencia que **nem toda variável inicialmente presente influencia o comportamento lógico final do sistema.**

No contexto da **lógica digital**, essa simplificação implica a eliminação completa da dependência da variável **B**, reduzindo o circuito a uma operação **OR** entre **A** e **C**. Na **programação**, equivale a remover condições irrelevantes de uma expressão lógica, tornando o código mais claro e eficiente.

3.4 Extração da expressão por mintermos e simplificação algébrica

A extração da expressão lógica por **mintermos** consiste em identificar, na tabela-verdade, todas as combinações de entradas para as quais a saída da função é igual a **1**. Cada uma dessas combinações gera um termo produto, formado pelas variáveis diretas ou complementadas conforme seus valores na linha correspondente.

A soma de todos esses termos forma a **expressão canônica** da função. Em seguida, essa expressão pode ser **simplificada algebricamente** por meio das leis da álgebra Booleana (absorção, distributiva, identidade e complementação), reduzindo o número de termos e variáveis, sem alterar o comportamento lógico do circuito.

3.4.1 Leitura da tabela-verdade e identificação das linhas relevantes

O ponto de partida é a tabela-verdade da função $X(A, B)$. No método dos **mintermos**, selecionamos **apenas as linhas em que a saída $X = 1$** , pois cada uma dessas linhas representa uma condição específica em que a função deve ser verdadeira.

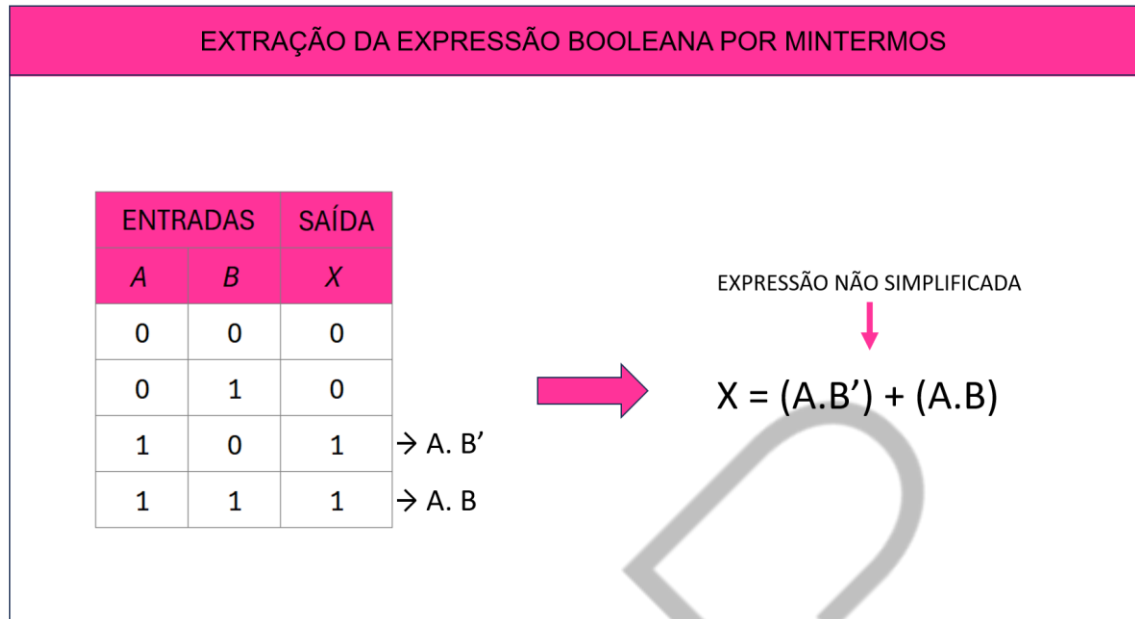


Figura 23 – Extração da expressão booleana por mintermos
 Fonte: Elaborado pelo autor (2026)

No exemplo, $X = 1$ ocorre nas combinações:

- $A = 1, B = 0$
- $A = 1, B = 1$

3.4.2 Construção dos mintermos (por que negar quando a entrada é 0?)

Cada **mintermo** deve “capturar” exatamente uma linha da tabela-verdade. Para isso:

- Quando a variável vale **1**, ela aparece **sem negação**;
- Quando a variável vale **0**, ela aparece **negada**.

A justificativa é funcional, a negação garante que o termo só será verdadeiro quando a variável assumir exatamente aquele valor da linha escolhida.

Aplicando ao exemplo:

- Para $(A = 1, B = 0)$:

$$M_2 = A \cdot B'$$

(usa-se B' porque $B = 0$ nessa linha)

- Para $(A = 1, B = 1)$:

$$M_3 = A \cdot B$$

EXTRAÇÃO DA EXPRESSÃO BOOLEANA POR MINTERMOS		
ENTRADAS		SAÍDA
A	B	X
0	0	0
0	1	0
1	0	1
1	1	1

$\rightarrow m_2 = A \cdot B'$
 $\rightarrow m_3 = A \cdot B$

$X = m_2 + m_3$
 $X = (A \cdot B') + (A \cdot B)$

Figura 24 – Extração da expressão booleana por mintermos, expressão completa
 Fonte: Elaborado pelo autor (2026)

3.4.3 Por que o operador AND é usado dentro do mintermo

Dentro de um mintermo, utilizamos **AND** (·) porque o termo representa uma **condição simultânea**: a linha da tabela só é representada corretamente se **todas** as variáveis assumirem aqueles valores ao mesmo tempo.

Exemplo: $A \cdot B'$ só é 1 quando $A = 1$ e $B = 0$ simultaneamente.

3.4.4 Formação da expressão final (soma de produtos)

Depois de construir todos os mintermos correspondentes às linhas em que $X = 1$, formamos a função fazendo o **OR** (+) entre eles, pois a saída deve ser 1 quando **qualquer uma** das condições verdadeiras ocorrer.

$$X = (A \cdot B') + (A \cdot B)$$

Essa forma é chamada de **Soma de Produtos (SOP)**:

- “Produtos” = termos com AND (mintermos);
- “Soma” = combinação dos termos com OR.

3.4.5 Simplificação algébrica da expressão obtida

$$\boxed{X = (A \cdot B') + (A \cdot B)}$$

Qualquer variável $\cdot 1 =$ Ela mesmo $\rightarrow X = (A \cdot B') + (A \cdot B) \xleftarrow{\text{Fatoração da Variável, A em evidencia}} X = A \cdot (1 \cdot B' + 1 \cdot B)$

$A \cdot 1 = A \rightarrow X = A \cdot (B' + B) \xleftarrow{B' + B = 1} X = A \cdot (1)$

$$X = A$$

Figura 25 – Simplificação algébrica da expressão $X=(A \cdot B')+(A \cdot B)$
Fonte: Elaborado pelo autor (2026)

Apresentaremos, de forma detalhada e justificativa, o passo a passo do processo de simplificação desta expressão.

Agora simplificamos a expressão extraída:

$$X = (A \cdot B') + (A \cdot B)$$

Passo 1 – Fatoração (colocar A em evidência):

A fatoração é útil porque evidencia padrões e permite aplicar leis fundamentais com mais clareza:

$$X = A \cdot (B' + B)$$

Passo 2 – Lei do complemento:

$$B' + B = 1$$

Intuitivamente: “B ou não-B” é sempre verdadeiro.

Logo:

$$X = A \cdot 1$$

Passo 3 – Lei da identidade (elemento neutro do AND):

$$A \cdot 1 = A$$

Portanto, a forma simplificada é:

$$\boxed{X = A}$$

3.4.6 Interpretação do resultado

O resultado $X = A$ indica que, apesar da tabela envolver B , **a variável B não influencia o valor final de X** . Em termos práticos:

- **Em circuitos digitais**, elimina-se a necessidade de portas associadas a B ;
- **Em programação**, remove-se uma condição redundante, tornando a lógica mais direta e legível.

3.5 Inter extração da expressão por maxtermos e simplificação algébrica (exemplo completo)

A extração da expressão lógica por **maxtermos** baseia-se na identificação, na tabela-verdade, de todas as combinações de entrada para as quais a saída da função é igual a **0**. Cada uma dessas combinações gera um termo soma, formado pelas variáveis diretas ou complementadas conforme seus valores na linha correspondente.

O produto de todos esses termos constitui a **expressão canônica na forma de produto de somas**. Assim como no método dos mintermos, essa expressão pode ser **simplificada algebricamente** por meio das propriedades da álgebra booleana, obtendo-se uma forma mais compacta e eficiente, sem modificar a função lógica original.

3.5.1 Leitura da tabela-verdade e identificação das linhas relevantes

No método dos **maxtermos**, o foco não está nas linhas em que $X = 1$, mas sim nas linhas em que a saída assume o valor **zero**. A ideia é descrever **todas as condições que tornam a função falsa** e, em seguida, combinar essas condições de modo a construir uma expressão que só seja verdadeira quando **nenhuma** dessas condições ocorrer.

Pela tabela-verdade do exemplo, $X = 0$ ocorre nas combinações:

- $A = 0, B = 0$
- $A = 0, B = 1$

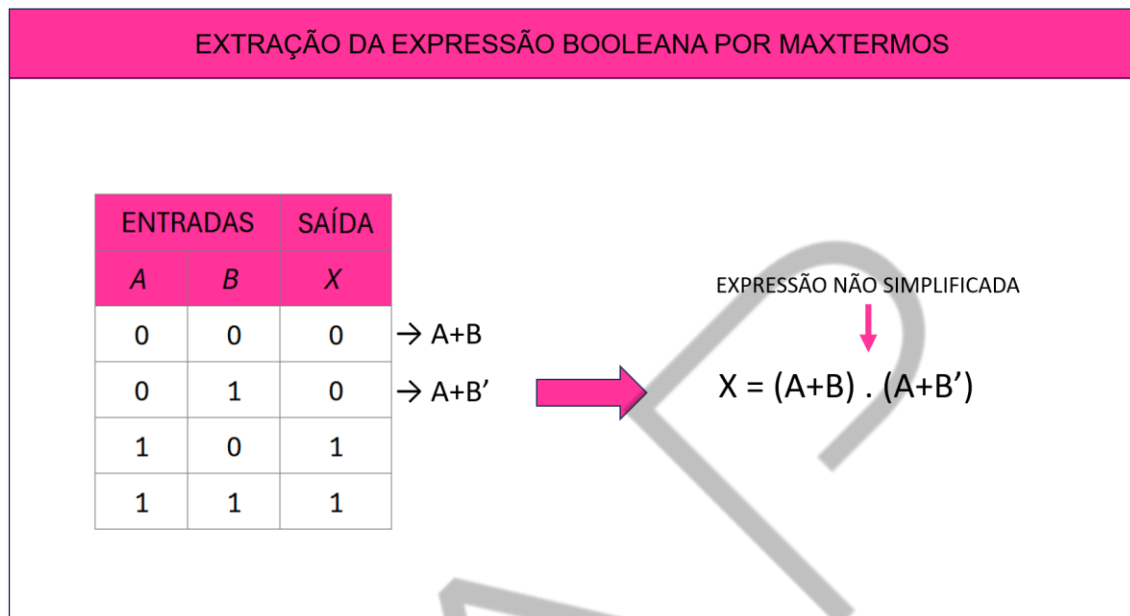


Figura 26 – Extração da expressão booleana por maxtermos (1)
 Fonte: Elaborado pelo autor (2026)

3.5.2 Construção dos maxtermos (por que negar quando a entrada é 1?)

Um **maxtermo** é construído para ser **igual a 0 exatamente na linha selecionada** (aquela em que $X = 0$) e igual a 1 nas demais. Para isso, utiliza-se a regra:

- Se a variável na linha vale **0**, ela entra **sem negação**;
- Se a variável na linha vale **1**, ela entra **negada**.

Essa escolha garante que, na linha alvo, a expressão interna do maxtermo se torne 0 (por ser uma soma/OR que falha exatamente ali).

Aplicando ao exemplo:

- Para $(A = 0, B = 0)$:

$$M_0 = (A + B)$$

(nenhuma variável é negada, pois ambas estão em 0)

- Para $(A = 0, B = 1)$:

$$M_1 = (A + B')$$

(B' aparece porque $B = 1$ na linha selecionada)

EXTRAÇÃO DA EXPRESSÃO BOOLEANA POR MAXTERMS

ENTRADAS		SAÍDA
A	B	X
0	0	0
0	1	0
1	0	1
1	1	1

→ m0 = A+B

→ m1 = A+B'

→

$$X = m0 + m1$$
$$X = (A+B) \cdot (A+B')$$

Figura 27 – Extração da expressão booleana por maxtermos (2)
Fonte: Elaborado pelo autor (2026)

3.5.3 Por que o operador OR é usado dentro do maxtermo

Dentro de um maxtermo usa-se **OR (+)** porque o objetivo é criar uma cláusula que seja **falsa (0) apenas em uma combinação específica**. Uma soma booleana $A + B$ só é 0 quando **ambas as entradas são 0**, assim, estruturando adequadamente as negações, conseguimos que cada maxtermo “zere” exatamente na linha desejada.

Exemplo:

- $(A+B)$ é 0 apenas quando $A = 0$ e $B = 0$.
- $(A+B')$ é 0 apenas quando $A = 0$ e $B' = 0$, isto é, $B = 1$.

3.5.4 Formação da expressão final (produto de somas)

Depois de construir todos os maxtermos associados às linhas em que $X = 0$, a expressão final é formada pelo operador **AND (·)**, pois queremos que a função seja

verdadeira somente quando **todas as restrições forem simultaneamente satisfeitas** (isto é, quando não estivermos em nenhuma linha que produz 0).

Logo, a função na forma canônica **Produto de Somas (POS)** é:

$$X = (A + B) \cdot (A + B')$$

Essa forma recebe o nome **produto de somas** porque:

- Cada maxtermo é uma **soma** (OR) de literais;
- Todos os maxtermos são combinados por um **produto** (AND).

3.5.5 Simplificação algébrica da expressão obtida

$$\begin{aligned}
 & \boxed{X = (A \cdot B) + (A \cdot B')} \\
 \text{Lei Distributiva} & \rightarrow X = (A \cdot B) + (A \cdot B') \\
 A \cdot A = A & \rightarrow X = A \cdot A + A \cdot B' + B \cdot A + B \cdot B' \\
 & \quad X = A + A \cdot B' + B \cdot A + B \cdot B' \quad \leftarrow B' \cdot B = 0 \\
 A + 0 = A & \rightarrow X = A + A \cdot B' + B \cdot A + 0 \\
 & \quad X = A + A \cdot B' + B \cdot A \quad \leftarrow \text{Fatoração da Variável, A em evidencia} \\
 & \quad X = A + A \cdot (B' + B \cdot 1) \\
 B' + B = 1 & \rightarrow X = A + A \cdot (B' + B) \\
 & \quad X = A + A \cdot (1) \quad \leftarrow A \cdot 1 = A \\
 A + A = A & \rightarrow X = A + A \\
 & \quad X = A
 \end{aligned}$$

Figura 28 – Simplificação algébrica da expressão $X=(A \cdot B) + (A \cdot B')$
 Fonte: Elaborado pelo autor (2026)

Apresentaremos, de forma detalhada e justificativa, o passo a passo do processo de simplificação desta expressão.

Agora simplificamos:

$$X = (A + B) \cdot (A + B')$$

Aplicando a distributiva:

$$X = A \cdot A + A \cdot B' + B \cdot A + B \cdot B'$$

Aplicando propriedades fundamentais:

- $A \cdot A = A$
- $A \cdot B' = A \cdot B'$
- $B \cdot A = A \cdot B$ (comutativa)
- $B \cdot B' = 0$ (complemento)

Logo:

$$X = A + A \cdot B' + A \cdot B + 0$$

Agrupando os termos com A (Fatorar):

$$X = A + A(B' + B)$$

Pela lei do complemento:

$$B' + B = 1$$

Então:

$$X = A + A \cdot 1$$

Como $A \cdot 1 = A$:

$$X = A + A$$

E pela idempotência:

$$A + A = A$$

Portanto:

$$\boxed{X = A}$$

3.5.6 Comparação final: igualdade entre as simplificações por mintermos e por maxtermos

Este exemplo evidencia um princípio central da lógica booleana: **a forma de extração (mintermos ou maxtermos) pode gerar expressões diferentes, mas ambas descrevem a mesma função**. Consequentemente, quando simplificadas corretamente, ambas devem conduzir ao **mesmo resultado final**, pois a função lógica subjacente é a mesma.

- Pela extração por **mintermos**, obteve-se uma expressão do tipo **Soma de Produtos (SOP)**, que ao simplificar resultou em:

$$X = A$$

- Pela extração por **maxtermos**, obteve-se uma expressão do tipo **Produto de Somas (POS)**, que ao simplificar também resultou em:

$$X = A$$

Assim, a igualdade das simplificações confirma que:

1. **Mintermos e maxtermos são representações canônicas equivalentes;**
2. A simplificação algébrica preserva a função original;
3. Apesar de caminhos algébricos distintos, o resultado simplificado deve ser idêntico quando ambas representam a mesma tabela-verdade.

Esse entendimento consolida a transição do capítulo para técnicas mais sistemáticas de simplificação, reforçando que a otimização altera a **forma** da expressão, mas nunca sua **função lógica**.

CONCLUSÃO

Ao longo deste capítulo, foi possível compreender a lógica booleana como um dos pilares fundamentais da ciência da computação, tanto no domínio teórico quanto em suas aplicações práticas em sistemas digitais e algoritmos. A relação entre expressões booleanas, tabelas-verdade e circuitos lógicos evidenciou como problemas do mundo real podem ser formalmente modelados e implementados de forma precisa e confiável.

O estudo das funções booleanas, das constantes $\{0,1\}$ e das propriedades da álgebra de Boole forneceu a base conceitual necessária para analisar e transformar expressões lógicas de maneira rigorosa. A introdução dos mintermos e maxtermos, bem como de suas formas canônicas, Soma de Produtos (SOP) e Produto de Somas (POS), demonstrou que uma mesma função pode ser representada por diferentes estruturas algébricas, todas logicamente equivalentes.

Por meio dos exemplos de simplificação, ficou evidente que expressões inicialmente extensas e redundantes podem ser reduzidas a formas mais simples, sem qualquer alteração no comportamento lógico. Essa otimização impacta diretamente a eficiência dos sistemas, reduzindo a complexidade de circuitos digitais e tornando expressões lógicas em programas mais claras, seguras e fáceis de manter.

Assim, a lógica booleana e a simplificação algébrica não devem ser vistas apenas como ferramentas matemáticas, mas como instrumentos essenciais para o desenvolvimento de soluções computacionais eficientes, robustas e conceitualmente corretas, preparando o estudante para desafios mais avançados no projeto de sistemas digitais e na construção de software.

REFERÊNCIAS

- BOOLE, George. **The Laws of Thought**. Cambridge: Macmillan, 1854.
- BROOKSHEAR, J. Glenn; BRYLOW, Dennis. **Ciência da Computação: Uma Visão Abrangente**. 12ª edição. Porto Alegre: Bookman, 2020.
- CRUM, Gary; RAYMOND, Sharon. **Digital Fundamentals**. 11ª edição. New York: Pearson, 2017.
- DEVLIN, Keith. **The Math Gene: How Mathematical Thinking Evolved and Why Numbers Are Like Gossip**. New York: Basic Books, 2000.
- FLOYD, Thomas L. **Digital Fundamentals**. 11ª edição. Upper Saddle River: Pearson, 2015.
- HALLIDAY, David; RESNICK, Robert; WALKER, Jearl. **Fundamentos de Física – Eletricidade e Magnetismo**. 10ª edição. Rio de Janeiro: LTC, 2016.
- HENNESSY, John L.; PATTERSON, David A. **Computer Organization and Design: The Hardware/Software Interface**. 5ª edição. Burlington: Morgan Kaufmann, 2014.
- KNUTH, Donald E. **The Art of Computer Programming – Fundamental Algorithms**. 3ª edição. Boston: Addison-Wesley, 1997.
- MANO, Morris M.; KIME, Charles R. **Lógica Digital e Projeto de Sistemas Digitais**. 5ª edição. São Paulo: Pearson, 2016.
- MENDONÇA, José Roberto. **Eletrônica Digital**. São Paulo: Érica, 2012.
- PATTERSON, David A.; HENNESSY, John L. **Organização e Projeto de Computadores**. 5ª edição. Rio de Janeiro: Elsevier, 2014.
- ROSEN, Kenneth H. **Matemática Discreta e Suas Aplicações**. 7ª edição. Porto Alegre: AMGH, 2012.
- SANTOS, José Plínio; MELLO, Edson F.; MURAKAMI, Carlos D. **Introdução à Lógica Matemática**. São Paulo: Cengage Learning, 2014.
- SILVA, Sérgio L. da. **Fundamentos de Sistemas Digitais**. São Paulo: Pearson, 2018.
- STALLINGS, William. **Computer Organization and Architecture: Designing for Performance**. 11ª edição. Boston: Pearson, 2019.
- TANENBAUM, Andrew S. **Structured Computer Organization**. 6ª edição. Upper Saddle River: Pearson, 2013.
- TUCKER, Allen B. **Computer Science Handbook**. 2ª edição. Boca Raton: Chapman & Hall/CRC, 2004.

WIRTH, Niklaus. **Algorithms + Data Structures = Programs**. Englewood Cliffs: Prentice-Hall, 1976.

EMAP

GLOSSÁRIO

Álgebra Boole	Ramo da matemática desenvolvido por George Boole que opera com valores binários (0 e 1) e operadores lógicos, servindo como base matemática da eletrônica digital e da computação.
AND (E)	Operação lógica representada por “ $\cdot$ ”, cujo resultado é 1 somente quando todas as entradas são 1.
Circuito lógico	Implementação física de uma expressão booleana por meio da interconexão de portas lógicas.
Complemento (negação)	Operação que inverte o valor lógico de uma variável, representada por A' .
Constantes booleanas	Valores fixos da lógica booleana: 0 (falso) e 1 (verdadeiro).
Disjunção (OR)	Operação lógica representada por “ $+$ ”, cujo resultado é 1 quando ao menos uma das entradas é 1.
Equivalência lógica	Condição em que duas expressões produzem os mesmos valores de saída para todas as combinações de entrada.
Expressão booleana	Representação algébrica de relações lógicas entre variáveis binárias.
Fatoração	Técnica algébrica que consiste em colocar termos comuns em evidência para facilitar a simplificação.

Função booleana	Relação matemática que associa um conjunto de entradas binárias a uma única saída binária.
Lei da dominação	Propriedade que define: $A + 1 = 1$ e $A \cdot 0 = 0$.
Lei da identidade	Propriedade que define: $A + 0 = A$ e $A \cdot 1 = A$.
Lei da idempotência	Propriedade que estabelece: $A + A = A$ e $A \cdot A = A$.
Lei do complemento	Propriedade que define: $A + A' = 1$ e $A \cdot A' = 0$.
Lei distributiva	Propriedade que permite distribuir AND sobre OR e vice-versa.
Maxtermo	Soma lógica que representa uma linha da tabela-verdade em que a saída da função é 0.
Mintermo	Produto lógico que representa uma linha da tabela-verdade em que a saída da função é 1.
NOT (NÃO)	Operação lógica que inverte o valor de uma variável.
OR (OU)	Operação lógica cujo resultado é 1 quando ao menos uma das entradas é 1.
Porta lógica	Componente eletrônico que implementa fisicamente uma operação booleana.
Precedência de operadores	Ordem de avaliação das operações: parênteses, AND e OR.

Produto de somas (POS)	Forma canônica em que a função booleana é expressa como AND entre maxtermos.
Simplificação algébrica	Processo de transformar uma expressão booleana em uma forma equivalente mais simples.
Soma de produtos (SOP)	Forma canônica em que a função booleana é expressa como OR entre mintermos.
Tabela-verdade	Representação tabular que mostra todas as combinações possíveis das entradas e a saída correspondente.
Termo booleano	Expressão formada por variáveis, constantes e operadores lógicos.
Variável booleana	Símbolo que representa uma condição lógica que pode assumir apenas os valores 0 ou 1.